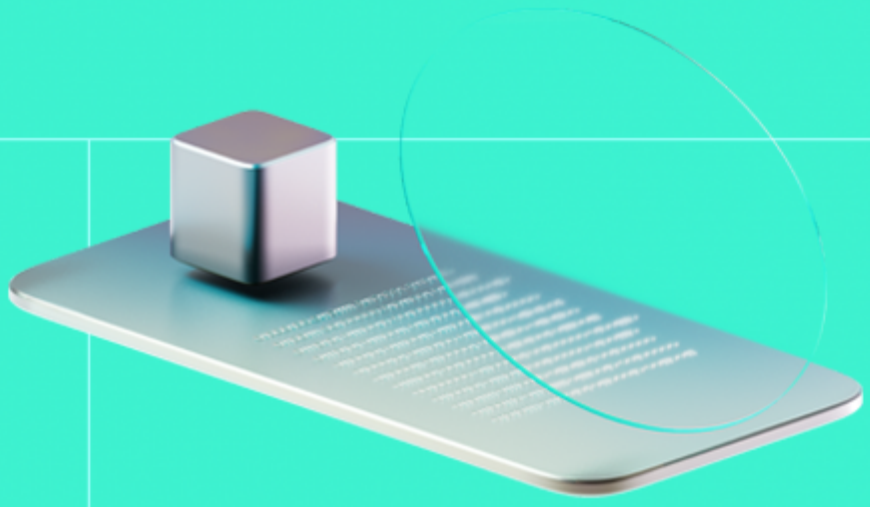
HACKEN

# Smart Contract Code Review And Security Analysis Report



**Customer:** RYT

**Date:** 13/01/2025

We express our gratitude to the RYT team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

RYT is a Layer-1 blockchain, built on its patented Proof of Majority (PoM) consensus mechanism, designed to address the core limitations that have held back blockchain from mainstream adoption: speed, scalability, accessibility, decentralization, and energy efficiency.

## Document

| | |
|---|---|
| Name | Smart Contract Code Review and Security Analysis Report for RYT |
| Audited By | Turgay Arda Usman, Viktor Lavrenenko |
| Approved By | Ataberk Yavuzer |
| Website | https://ryt.io/ |
| Changelog | 16/12/2025 - Preliminary Report |
| | 13/01/2025 - Final Report |
| Platform | RYT blockchain |
| Language | Solidity |
| Tags | Fungible Token ; Staking |
| Methodology | https://docs.hacken.io/methodologies/smart-contracts |

## Review

## Scope

| | |
|---|---|
| 1st Repository | https://github.com/ryt-io/Komitte-Contract |
| Initial Commit | 52685f90e291cc6e9be3a5a4bba052fe2f940164 |
| Final Commit | 1829f31e73312ff587f102cc2246d3a646982195 |
| 2nd Repoitory | ryt-stablecoin-develop.zip |
| Initial fille SHA3-256 | 851798d95a8fa81a37c950c9c2da0a4085a96f52e73d38eb2f044b56868adb21 |

## Review

## Scope

| | |
|---|---|
| Final fille<br><br>SHA3-256 | 04264b6db0b5fdf81a83d00c9cd685e2d715a9af6ebdc33d30c95755e006ac56 |

## Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

| 31 | 25 | 6 | 0 |
|---|---|---|---|
| Total Findings | Resolved | Accepted | Mitigated |

## Findings by Severity

| Severity | Count |
|---|---|
| Critical | 1 |
| High | 4 |
| Medium | 5 |
| Low | 7 |

| Vulnerability | Severity | Status |
|---|---|---|
| F-2025-14249 - Primary Contributor Deposits Are Not Recorded, Leading to Permanent Fund Loss and Incorrect Payouts | Critical | Fixed |
| F-2025-14222 - Security Mechanisms Inoperative due to OpenZeppelin v5 Hook Incompatibility | High | Fixed |
| F-2025-14250 - Winner-Selection Logic Flaw Allows The Group Creator To Capture All Contributed Funds | High | Fixed |
| F-2025-14265 - Winner Selection Ignores Assigned Payout Positions Due To A Faulty If Condition | High | Fixed |
| F-2025-14280 - Secondary Contributions Not Recorded in Slot Total, Leading to Incorrect Payout Ratios | High | Fixed |
| F-2025-14229 - Missing USER_ROLE Check Allows Unauthorized Participants in Joint Groups And Payouts | Medium | Fixed |
| F-2025-14233 - Broken Participant Invariant Due to Unrestricted USER_ROLE Revocation | Medium | Accepted |
| F-2025-14253 - Missing Refund Mechanism for Regular Members Leads To Stuck Assets | Medium | Fixed |
| F-2025-14273 - Excess Contributions Become Permanently Locked Due to Non-Exact Deposit Enforcement | Medium | Fixed |
| F-2025-14287 - DoS via Concurrent Joint Contributor Invites | Medium | Fixed |
| F-2025-14224 - Weak RNG: block.timestamp Enables Predictable and Manipulable Payout Shuffling | Low | Accepted |
| F-2025-14226 - Missing startTime Validation Enables Creation of Invalid Groups That Fail Existence Checks | Low | Fixed |

| Vulnerability | Severity | Status |
|---|---|---|
| F-2025-14230 - Missing Primary Contributor Validation Enables Uninvited Users to Join Groups | Low | Fixed |
| F-2025-14267 - Unset Payout Positions Default to Index 0, Allowing Any Unpaid Member to Illegitimately Win First Payout | Low | Accepted |
| F-2025-14285 - Protocol Insolvency Due to Premature Payout Triggering | Low | Fixed |
| F-2025-14286 - Missing Minimum Slot Validation in updateKomiti() Allows Bypassing Group Size Invariant | Low | Fixed |
| F-2025-14306 - Refund Logic Becomes Irreversible After Invitation Acceptance, Leading to Fund Lock for Both Joint Contributors | Low | Fixed |
| F-2025-14219 - Presence of Debugging and Commented Code in Production Contract | Info | Fixed |
| F-2025-14220 - Risks from Using Outdated Solidity and OpenZeppelin Libraries | Info | Accepted |
| F-2025-14221 - Use of transfer() instead of call() to send native tokens | Info | Fixed |
| F-2025-14223 - Missing Empty-Length Array Validation in Loop-Based Functions | Info | Fixed |
| F-2025-14225 - Missing Zero Address Validation | Info | Fixed |
| F-2025-14227 - Incorrect Modifier Naming Reduces Code Clarity and Increases Risk of Misuse | Info | Fixed |
| F-2025-14234 - Absence of Custom Errors Leading to Increased Gas Costs | Info | Fixed |
| F-2025-14258 - Violation of Solidity Naming Conventions in Events Reduces Code Quality and Readability | Info | Fixed |
| F-2025-14293 - Integer Division Precision Loss Causes Underfunded Joint Contributions | Info | Fixed |
| F-2025-14299 - Lack of Mutual Exclusion Allows Role Overlap Risks | Info | Accepted |
| F-2025-14300 - Operational Lockout Risk Due to Missing Initial Manager Assignment | Info | Fixed |
| F-2025-14301 - Floating Pragma | Info | Fixed |
| F-2025-14302 - Redundant Imports and Unused Error Definitions Increase Gas Usage | Info | Fixed |
| F-2025-14305 - Unchecked Return Values From Functions May Cause Silent State Inconsistencies | Info | Accepted |

## Documentation quality

- Functional requirements are not complete:
  - The project's purpose is  described.
  - Business logic is provided.
  - Use cases are partially provided: pausing flows are not described.
  - Project's features are described.
- Technical description is sufficient:
  - Key function descriptions are provided for the files in the scope.
  - Architectural overview is missing.
  - Roles and authorization are mentioned.
  - Information on used technologies provided.

## Code quality

- The code partially follows best practices and Solidity Style Guide.
  - See informational findings for more details.
- The development environment is configured.

## Test coverage

Code coverage of the `Komiti` project is **96.27%** (branch coverage).

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is partially missed.
- Interactions by several users are tested thoroughly.

Code coverage of the `RYTStablecoin` project is **22.41 %** (branch coverage).

- Deployment and basic user interactions are not covered with tests.
- Negative cases coverage is missed.
- Interactions by several users are not tested thoroughly.

# Table of Contents

## System Overview

`Komiti` - is a decentralized group savings and contribution contract that allows users to form groups with predetermined rules for contributions and payouts. The contract supports both randomized and sequential payout mechanisms, as well as joint contributor arrangements.

`RYTStablecoin` - an ERC20-based stablecoin with role-based access control, blocklisting, pausing, and rescue functionality.

## Privileged roles

- The `DEFAULT_ADMIN_ROLE` role of the `Komiti` contract can assign new roles via `grantRole()` and revoke via `revokeRole()` functions.
- The `ADMIN_ROLE` role can:
  - create new groups via `createGroup()`.
  - update the existing group via `updateKomiti()`.
  - start the payout process of the group via `startKomiti()`.
  - start the distribution of funds via `distributeFunds()`.
  - pause/unpause the contract via `triggerJointContribution()`.
- The `USER_ROLE` role can:
  - join a group via `joinGroup()`.
  - join a group without a joint contributor via `joinGroupWithJointContributor()`.
  - accept the invite for the joint contributor via `acceptInviteForJointContributor()`.
  - return the funds via `returnFunds()`.
  - contribute to the group via `contribute()`.
  - contribute to the group as a joint member via `contributeAsJointMember()`.
- The `MANAGER_ROLE` role can:
  - assign/revoke `USER_ROLE` roles via `assignUserRoles()` and `revokeUserRoles()`.
  - assign/revoke `ORGANIZER_ROLE` roles via `assignOrganizerRoles()` and `revokeOrganizerRoles()`.
- The `ORGANIZER_ROLE` role can create new groups via `createGroup()`.
- The `DEFAULT_ADMIN_ROLE` role of the `RYTStablecoin` contract can assign new roles via `grantRole()` and revoke via `revokeRole()` functions.
- The `MINTER_ROLE` role can mint new `RYTStablecoin` tokens via the `mint()` function.
- The `PAUSER_ROLE` role can pause/unpause the token functionality via `pause/unpause()` functions.
- The `BLOCKLISTER_ROLE` role can add/remove from the blacklist via `addToBlocklist()/removeFromBlocklist()` functions.
- The `RESCUER_ROLE` role can rescue the stuck tokens from the token contract via `rescueTokens()` function.

## Potential Risks

- The broad authorization model increases the risk of protocol control loss if any authorized address is compromised, potentially leading to unauthorized actions and significant financial loss.
- The digital contract architecture relies on administrative keys for critical operations. Centralized control over these keys presents a significant security risk, as compromise or misuse can lead to unauthorized actions or loss of funds.
- The project iterates over large dynamic arrays, which leads to excessive gas costs, risking denial of service due to out-of-gas errors, directly impacting contract usability and reliability.
- The project's smart-contracts are non-upgradable. While this approach enhances immutability, it also limits the ability to patch vulnerabilities, fix bugs, or introduce improvements after deployment. In the event of unintended behavior or security issues, the absence of an upgrade mechanism significantly reduces the available response options, potentially leading to prolonged downtime or the need for a full contract migration.
- The `Komiti` smart-contract utilizes the `totalSlots` to limit the number of users within a specific group. If the number is not properly set or adjusted and if the total number of members exceeds this limit, there is a risk of contract's misbehavior and locked funds.
- The lack of multi-signature requirements for key operations centralizes decision-making power, increasing vulnerability to single points of failure or malicious insider actions, potentially leading to unauthorized transactions or configuration changes.

# Findings

## Vulnerability Details

### F-2025-14249 - Primary Contributor Deposits Are Not Recorded, Leading to Permanent Fund Loss and Incorrect Payouts - Critical

**Description:**

The Komiti contract allows users to join a ROSCA (Rotating Savings and Credit Association) group individually or jointly. In the joint model (`joinGroupWithJointContributor`), a "primary" contributor invites a "secondary" contributor. Both parties contribute half of the required share amount. The contract holds these funds and later distributes the payout to the primary contributor, who is then responsible (via the contract's logic) for splitting the funds with the secondary contributors.

A logic error exists in the `joinGroupWithJointContributor` function where the primary contributor's deposit is accepted by the contract but not recorded in the internal accounting mapping (`s_contributions`). This omission causes two severe consequences: the primary contributor cannot retrieve their funds if they cancel the invitation (resulting in permanent fund loss), and during payouts, the secondary contributor receives 100% of the funds while the primary contributor receives zero.

The vulnerability stems from a commented-out line of code in the `joinGroupWithJointContributor` function. When a user calls this function, they must send native coins (`msg.value`) covering half the share amount. However, the line responsible for updating the `s_contributions` mapping is inactive.

```
function joinGroupWithJointContributor(uint256 groupId, address secondaryCont
ributor)
   external
   payable
   …
{
   require(msg.value >= group.perShareAmount / 2, "Komiti: Incorrect amount s
ent");
   …
   // Not needed
   // group.contributions[msg.sender] += msg.value;
   …
}
```

Because the state update is missing:

- **Refund Failure**: The `returnFunds` function relies on `s_contributions[groupId][msg.sender]` to determine how much to refund. Since this value remains 0 despite the user sending ETH, the function transfers 0 back to the user, leaving the actual ETH trapped in the contract.

```
function returnFunds(...) external {
      ...
      uint256 amountToReturn = s_contributions[groupId][msg.sender];
      s_contributions[groupId][msg.sender] = 0;
      (bool success,) = msg.sender.call{value: amountToReturn}("");
  }
```

- **Payout Calculation Failure**: The `distributeJointPayout` function calculates the secondary's share based on `totalContributions` (which reads from `s_contributions[primary]`).

```
    uint256 totalContributions = s_contributions[groupId][primaryContributor]
;
    ...
    uint256 secondaryShare = (totalAmount * secondaryContribution) / totalCon
tributions;
```

If the primary's contribution is missing, `totalContributions` will equal only the secondary's contribution (recorded in `acceptInviteForJointContributor`). Consequently, `secondaryContribution / totalContributions` becomes 1, awarding the entire payout to the secondary contributor.

This issue leads to direct and permanent loss of user funds.

1. **Theft of Deposit**: Any user who initiates a joint contribution and subsequently tries to cancel it (e.g., because the invitee declined) will lose their entire deposit. The funds remain locked in the contract indefinitely.
2. **Unfair Payout Distribution**: In a successful group cycle, the primary contributor effectively forfeits their share of the payout. The secondary contributor unknowingly "steals" the primary's portion, receiving double their intended return, while the primary contributor receives nothing.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

## Classification

| | |
|---|---|
| **Impact Rate:** | 5/5 |
| **Likelihood Rate:** | 5/5 |
| **Exploitability:** | Independent |
| **Complexity:** | Simple |
| **Severity:** | Critical |

## Recommendations

**Remediation:** Enable the recording of the primary contributor's deposit within the `joinGroupWithJointContributor` function by actively updating the `s_contributions` mapping with the received `msg.value`. This change ensures that funds are strictly accounted for, allowing users to retrieve their deposits upon cancellation and guaranteeing the payout logic correctly splits funds between joint contributors.

**Resolution:** Fixed in `1829f31`. The functions `joinGroupWithJointContributor()` and `acceptInviteForJointContributor()` have been removed from the codebase.

## Evidences

### PoC

**Reproduce:**

1. **User1** calls joinGroupWithJointContributor with 0.5 ETH (half share).
2. It verifies that getMemberInfoByGroup shows contributedAmount as **0**, confirming the state update was missed.
3. It calls returnFunds to cancel the invite.
4. It asserts that **User1** received **0 ETH** back (balance only decreased due to gas), proving the funds were not returned.
5. It asserts that the contract balance remains unchanged, proving the funds are permanently stuck in the contract.

PoC Code:

```
describe("Exploit: Permanent Fund Loss in joinGroupWithJointContributor", asy
nc () => {
    it("Should fail to refund primary contributor due to missing contribution
record", async () => {
    const halfShare = perShareAmount.div(2);

    // 1. Primary contributor joins with ETH
    await komiti.connect(user1).joinGroupWithJointContributor(groupId, user2.
address, {
```

```
    value: halfShare
    });

    // 2. Verify contribution was NOT recorded (Root Cause)
    const memberInfo = await komiti.getMemberInfoByGroup(groupId, user1.addre
ss);
    expect(memberInfo.contributedAmount).to.equal(0);

    // 3. Verify Contract holds the funds
    const contractBalanceBefore = await ethers.provider.getBalance(komiti.add
ress);

    // 4. Primary contributor tries to return funds (cancel invite)
    const userBalanceBeforeRefund = await user1.getBalance();

    const tx = await komiti.connect(user1).returnFunds(groupId, user2.address
);
    await tx.wait();

    const userBalanceAfterRefund = await user1.getBalance();

    // 5. Verify User received 0 ETH back (minus gas)
    const diff = userBalanceAfterRefund.sub(userBalanceBeforeRefund);
    expect(diff).to.be.lt(0);

    // 6. Verify funds are still stuck in contract
    const contractBalanceAfter = await ethers.provider.getBalance(komiti.addr
ess);
    expect(contractBalanceAfter).to.equal(contractBalanceBefore);
    });
});
```

**Results:**

```
  Exploit: Permanent Fund Loss in joinGroupWithJointContributor
      ✔ Should fail to refund primary contributor due to missing contribution
record
```

## F-2025-14222 - Security Mechanisms Inoperative due to OpenZeppelin v5 Hook Incompatibility - High

**Description:**

The `RYTStablecoin` contract is an `ERC20` token that implements regulatory and security compliance features. It relies on the `_beforeTokenTransfer` internal function to enforce two critical restrictions before any tokens are moved (minted, burned, or transferred):

1. **Pausability:** Transfers should revert if the contract is paused by an admin.
2. **Blocklisting:** Transfers should revert if the sender or receiver is in the isBlocklisted mapping.

The project dependencies include OpenZeppelin Contracts `v5.4.0`, but the contract implements the `_beforeTokenTransfer` hook, which was removed in version 5.0 and replaced by `_update`. Because the parent `ERC20` implementation in v5 never calls `_beforeTokenTransfer`, the custom security logic in `RYTStablecoin` is effectively dead code. Consequently, the Pause and Blocklist features are completely non-functional, allowing blocked users to transfer funds and preventing the admin from halting the contract during emergencies.

In OpenZeppelin Contracts `v5.0`, the `_beforeTokenTransfer` and `_afterTokenTransfer` hooks were refactored into a single `_update` function. The `RYTStablecoin.sol` contract defines its security logic as follows:

```
function _beforeTokenTransfer(address from, address to, uint256 amount)
internal
whenNotPaused
{
    // Skip checks for zero amount to allow no-op approvals/permits with tran
sfer hooks elsewhere
    if (amount == 0) return;

    if (from == address(0)) {
        // Mint
        if (isBlocklisted[to]) revert Blocklisted(to);
    } else if (to == address(0)) {
        // Burn
        if (isBlocklisted[from]) revert Blocklisted(from);
    } else {
        if (isBlocklisted[from]) revert Blocklisted(from);
        if (isBlocklisted[to]) revert Blocklisted(to);
```

```
    }
}
```

However, the installed version of OpenZeppelin (`v5.4.0`) implements token transfers in the base ERC20 contract by calling `_update`. It does **not** look for or execute `_beforeTokenTransfer`. Since `RYTStablecoin` does not override `_update`, the base implementation is used, which performs the transfer without performing any of the checks defined in `RYTStablecoin`. This results in a state where:

1. `_pause()` updates the pause state variable, but the `whenNotPaused` modifier on `_beforeTokenTransfer` is never triggered.
2. `addToBlocklist()` updates the mapping, but the check inside `_beforeTokenTransfer` is never executed.

This vulnerability renders the contract's primary security features useless:

1. **Inability to Pause:** In the event of a bridge hack, private key compromise, or critical bug, the `PAUSER_ROLE` cannot stop token transfers. This could lead to the complete draining of liquidity pools or theft of user funds.
2. **Bypass of Blocklist:** Malicious actors, sanctioned entities, or hackers identified by the `BLOCKLISTER_ROLE` can still freely mint, burn, and transfer tokens. This causes the token to fail specific regulatory compliance requirements and prevents the freezing of stolen funds.

**Assets:**

- contracts/RYTStablecoin.sol [ryt-stablecoin-develop.zip]

**Status:** Fixed

## Classification

**Impact Rate:** 4/5

**Likelihood Rate:** 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** High

## Recommendations

**Remediation:** Refactor the contract to use the OpenZeppelin v5 `_update` hook instead of `_beforeTokenTransfer`.

**Resolution:** In file hash `851798d95a8fa81a37c950c9c2da0a4085a96f52e73d38eb2f044b56868adb21`, the fix replaces the deprecated `_beforeTokenTransfer` hook with the correct `_update` override, ensuring that pausability and blocklisting checks are actually executed during all token transfers, mints, and burns.

## Evidences

### PoC #1

**Reproduce:**

1. **Setup**: Mints 1000 tokens to a user.
2. **Action**: Pauses the contract using token.pause().
3. **Exploit**: Attempts to transfer tokens from the user to the deployer while the contract is paused.
4. **Assertion**: Verifies that the transfer **succeeded** (checking the balance updated), proving that the Pause mechanism is ignored because _beforeTokenTransfer is never called.

POC CODE:

```
it("SECURITY FLAW PROOF: Transfers succeed even when Paused (Hooks are dead)"
, async function () {
    // 1. Mint to user
    await token.mint(user.address, 1000);

    // 2. Pause contract
    await token.pause();
    expect(await token.paused()).to.equal(true);

    // 3. Attempt transfer (Should revert if fixed, but succeeds now)
    await token.connect(user).transfer(deployer.address, 100);

    // 4. Assert transfer happened (proving the bug)
    expect(await token.balanceOf(deployer.address)).to.equal(100);
});
```

**Results:**

```
RYTStablecoin
    ✔ should mint tokens within cap
    ✔ SECURITY FLAW PROOF: Transfers succeed even when Paused (Hooks are dead
) (65ms)
    ✔ SECURITY FLAW PROOF: Blocklisted addresses can still transfer (Hooks ar
e dead) (39ms)
```

### PoC #2

**Reproduce:**

1. **Setup**: Mints tokens to a user.
2. **Action**: Adds the user to the blocklist.
3. **Exploit**: Attempts a transfer from the blocklisted user.
4. **Assertion**: Verifies the transfer **succeeded**, confirming that _beforeTokenTransfer is ignored and the blocklist is not enforced.

POC CODE:

```
it("SECURITY FLAW PROOF: Blocklisted addresses can still transfer (Hooks are dead)", async function () {
    // 1. Mint to user
    await token.mint(user.address, 1000);

    // 2. Blocklist the user
    await token.addToBlocklist(user.address);
    expect(await token.isBlocklisted(user.address)).to.equal(true);

    // 3. Attempt transfer from blocklisted user (Should revert if fixed, but succeeds now)
    await token.connect(user).transfer(deployer.address, 100);

    // 4. Assert transfer happened (proving the bug)
    expect(await token.balanceOf(deployer.address)).to.equal(100);
});
```

**Results:**

```
RYTStablecoin
    ✔ should mint tokens within cap
    ✔ SECURITY FLAW PROOF: Transfers succeed even when Paused (Hooks are dead) (65ms)
    ✔ SECURITY FLAW PROOF: Blocklisted addresses can still transfer (Hooks are dead) (39ms)
```

## F-2025-14250 - Winner-Selection Logic Flaw Allows The Group Creator To Capture All Contributed Funds - High

**Description:**

The `distributeFunds()` function iterates through all group members, starting from index 0, to determine the payout winner. Since the `ADMIN/ORGANIZER` is always the **first member** inserted into the members array, and the selection logic favors the first unpaid member by default, the `ADMIN/ORGANIZER` is **always chosen as the first payout recipient**, even when they are not assigned a payout position, since the default/unset payout position equals to the initial `currentPayoutIndex` and is zero.

```
...
 for (uint256 i = 0; i < totalMembers; i++) {
            address member = selectedGroup.members[i];
            if (
                !s_hasReceivedPayout[groupId][member]
                    && (winner == address(0) || s_payoutPositions[groupId][me
mber] == selectedGroup.currentPayoutIndex)
            ) {
                winner = member;
                break;
            }
}
...
```

As a result, the payout logic becomes corrupted and allows the `ADMIN/ORGANIZER` to illegitimately win and receive all pooled funds during the first distribution.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 5/5

**Likelihood Rate:** 5/5

**Exploitability:** Semi-Dependent

**Complexity:** Medium

**Severity:** High

## Recommendations

**Remediation:** It is recommended to start the looping process from i = 1 (first index) to ensure that the first member of the group(i=0) is not included in the selection.

```
...
        for (uint256 i = 1; i < totalMembers; i++) {
            address member = selectedGroup.members[i];
            if (
                !s_hasReceivedPayout[groupId][member]
                    && (winner == address(0) || s_payoutPositions[groupId][me
mber] == selectedGroup.currentPayoutIndex)
            ) {
                winner = member;
                break;
            }
        }
...
```

**Resolution:** In commit `1829f31`, the `winner == address(0)` condition is removed from the loop, ensuring the winner is selected strictly by matching their assigned `s_payoutPositions` index rather than automatically defaulting to the first member (`Admin`) found in the list.

## Evidences

### POC

**Reproduce:**

1. `ADMIN` creates a new group with a sequential distribution process.
2. `USER1`, `USER2` and `USER3` join the group via `joinGroup()` function and contribute 0.1 ether each.
3. `ADMIN` starts the Komiti by calling the `startKomiti()` function.
4. `ADMIN` sets the payout positions via `setPayoutPositions()` function and excludes themselves.
5. `ADMIN` initiates the distribution process and calls `distributeFunds()`.
6. `ADMIN` is always the first winner of the selection group, even though they have not been added to the payouts position.

```
function testPOC2() public {
        testGroupCreationSequantial();
        uint256 groupId = komiti.s_groupCounter();

        console.log("----------- USERS JOINING THE GROUP ----------");
```

```
        vm.startPrank(USER1);
        vm.deal(USER1, 1 ether);
        komiti.joinGroup{value: 0.1 ether}(groupId);
        vm.stopPrank();

        vm.startPrank(USER2);
        vm.deal(USER2, 1 ether);
        komiti.joinGroup{value: 0.1 ether}(groupId);
        vm.stopPrank();

        vm.startPrank(USER3);
        vm.deal(USER3, 1 ether);
        komiti.acceptInviteForJointContributor{value: 0.1 ether}(groupId, add
ress(0));
        vm.stopPrank();

        vm.warp(block.timestamp + 4 days); // to finish the join period

        //showGroupData(groupId);
        //funData(groupId, USER1, address(0));
        console.log("------------------------------------------------------------
------------------------------------------------------------");
        //funData(groupId, USER2, address(0));

        // start komiti
        vm.startPrank(ADMIN);
        komiti.startKomiti(komiti.s_groupCounter());
        vm.stopPrank();



        // set payout positions
        vm.startPrank(ADMIN);
        address [] memory payoutUsers = new address[](3);
        payoutUsers[0] = USER1;
        payoutUsers[1] = USER2;
        payoutUsers[2] = USER3;
        komiti.setPayoutPositions(groupId, payoutUsers);

        console.log("PAYOUT_POSITION: ", komiti.getPayoutPosition(groupId, US
ER1));
        console.log("PAYOUT_POSITION: ", komiti.getPayoutPosition(groupId, US
ER2));
        console.log("PAYOUT_POSITION: ", komiti.getPayoutPosition(groupId, US
ER3));
        console.log("ADMIN PAYOUT_POSITION: ", komiti.getPayoutPosition(group
Id, ADMIN));
        vm.stopPrank();

        // contributions
```

```
        // vm.startPrank(USER1);
        // komiti.contribute{value: 0.9 ether}(groupId);
        // vm.stopPrank();

        // vm.startPrank(USER2);
        // komiti.contribute{value: 0.9 ether}(groupId);
        // vm.stopPrank();

        // Distribution of funds
        vm.startPrank(ADMIN);
        komiti.distributeFunds(groupId);
        vm.stopPrank();

        // check if the funds are distributed
        console.log("FUNDS DISTRIBUTED: ", komiti.s_hasReceivedPayout(groupId
, USER1));
        console.log("FUNDS DISTRIBUTED: ", komiti.s_hasReceivedPayout(groupId
, USER2));
        console.log("FUNDS DISTRIBUTED: ", komiti.s_hasReceivedPayout(groupId
, ADMIN));
        console.log("FUNDS DISTRIBUTED: ", komiti.s_hasReceivedPayout(groupId
, USER3));
    }
```

**Files:** KomitiADMINPoc.t.sol

## F-2025-14265 - Winner Selection Ignores Assigned Payout Positions Due To A Faulty If Condition - High

**Description:**

The payout selection mechanism in `distributeFunds()` is intended to enforce a strict order - either sequential or randomized via the `s_payoutPositions` mapping and `setPayoutPositions()` function.

function `distributeFunds()`

```
...
for (uint256 i = 0; i < totalMembers; i++) {
            address member = selectedGroup.members[i];
            if (
                !s_hasReceivedPayout[groupId][member]
                    && (winner == address(0) || s_payoutPositions[groupId][me
mber] == selectedGroup.currentPayoutIndex)
            ) {
                winner = member;
                break;
            }
}
...
```

However, due to a flawed conditional structure , the selection logic **always picks the first unpaid member in the group**, completely ignoring the position assigned in `s_payoutPositions` via the `setPayoutPositions()` function. The `winner == address(0)` condition is always true at the start of each payout round, otherwise the looping is over and the winner is chosen. The winner condition is evaluated **before** checking whether the user has the correct payout position. This effectively short-circuits the entire payout-positioning system. As a result, the `s_payoutPositions` mapping has **little to no impact** on the selection process, violating the core invariant that the payout must follow the configured order.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 5/5

**Likelihood Rate:** 5/5

**Exploitability:** Semi-Dependent

**Complexity:** Medium

**Severity:** High

## Recommendations

**Remediation:** It is recommended to modify the current selection process and remove the `winner == address(0)` condition to ensure that the `s_payoutPositions` mapping has an effect on the selection process as follows:

```
...
for (uint256 i = 0; i < totalMembers; i++) {
            address member = selectedGroup.members[i];
            if (
                !s_hasReceivedPayout[groupId][member]
                    && (s_payoutPositions[groupId][member] == selectedGroup.c
urrentPayoutIndex)
            ) {
                winner = member;
                break;
            }
}
...
```

**Resolution:** In commit `1829f31`, the `distributeFunds` function now strictly selects the winner by matching their assigned payout position against the current cycle index, having removed the `winner == address(0)` condition that previously allowed the first unpaid member to bypass the ordering logic.

## Evidences

### POC

**Reproduce:**

1. `ADMIN` creates a new group with a sequential distribution process.
2. `USER1`, `USER2` and `USER3` join the group via `joinGroup()` function and contribute 0.1 ether each.
3. `ADMIN` starts the Komiti by calling the `startKomiti()` function.
4. `ADMIN` sets the payout positions via `setPayoutPositions()` function and excludes themselves.
5. `ADMIN` initiates the distribution process and calls `distributeFunds()`.
6. `ADMIN` is the winner of the selection group, since the payouts positions are not taken into account.

```
function testPOC2() public {
        testGroupCreationSequantial();
        uint256 groupId = komiti.s_groupCounter();

        console.log("----------- USERS JOINING THE GROUP ----------");
        vm.startPrank(USER1);
        vm.deal(USER1, 1 ether);
        komiti.joinGroup{value: 0.1 ether}(groupId);
        vm.stopPrank();

        vm.startPrank(USER2);
        vm.deal(USER2, 1 ether);
        komiti.joinGroup{value: 0.1 ether}(groupId);
        vm.stopPrank();

        vm.startPrank(USER3);
        vm.deal(USER3, 1 ether);
        komiti.acceptInviteForJointContributor{value: 0.1 ether}(groupId, add
ress(0));
        vm.stopPrank();

        vm.warp(block.timestamp + 4 days); // to finish the join period

        //showGroupData(groupId);
        //funData(groupId, USER1, address(0));
        console.log("-------------------------------------------------------
-------------------------------------------------------------");
        //funData(groupId, USER2, address(0));

        // start komiti
        vm.startPrank(ADMIN);
        komiti.startKomiti(komiti.s_groupCounter());
        vm.stopPrank();



        // set payout positions
        vm.startPrank(ADMIN);
        address [] memory payoutUsers = new address[](3);
        payoutUsers[0] = USER1;
        payoutUsers[1] = USER2;
        payoutUsers[2] = USER3;
        komiti.setPayoutPositions(groupId, payoutUsers);

        console.log("PAYOUT_POSITION: ", komiti.getPayoutPosition(groupId, US
ER1));
        console.log("PAYOUT_POSITION: ", komiti.getPayoutPosition(groupId, US
ER2));
        console.log("PAYOUT_POSITION: ", komiti.getPayoutPosition(groupId, US
```

```
ER3));
        console.log("ADMIN PAYOUT_POSITION: ", komiti.getPayoutPosition(group
Id, ADMIN));
        vm.stopPrank();

        // contributions
        // vm.startPrank(USER1);
        // komiti.contribute{value: 0.9 ether}(groupId);
        // vm.stopPrank();

        // vm.startPrank(USER2);
        // komiti.contribute{value: 0.9 ether}(groupId);
        // vm.stopPrank();

        // Distribution of funds
        vm.startPrank(ADMIN);
        komiti.distributeFunds(groupId);
        vm.stopPrank();

        // check if the funds are distributed
        console.log("FUNDS DISTRIBUTED: ", komiti.s_hasReceivedPayout(groupId
, USER1));
        console.log("FUNDS DISTRIBUTED: ", komiti.s_hasReceivedPayout(groupId
, USER2));
        console.log("FUNDS DISTRIBUTED: ", komiti.s_hasReceivedPayout(groupId
, ADMIN));
        console.log("FUNDS DISTRIBUTED: ", komiti.s_hasReceivedPayout(groupId
, USER3));
    }
```

**Results:**

```
 PAYOUT_POSITION:  0
PAYOUT_POSITION:  1
PAYOUT_POSITION:  2
ADMIN PAYOUT_POSITION:  0
FUNDS DISTRIBUTED:  false
FUNDS DISTRIBUTED:  false
FUNDS DISTRIBUTED:  true
FUNDS DISTRIBUTED:  false
```

**Files:** KomitiADMINPoc.t.sol

## F-2025-14280 - Secondary Contributions Not Recorded in Slot Total, Leading to Incorrect Payout Ratios - High

**Description:**

The `Komiti` contract supports "Joint Contributors," where two users share a single slot. The Primary Contributor's address is used to track the total "Slot Balance" in the `s_contributions` mapping. The Secondary Contributor's specific equity is tracked in `s_contributionShares`. When a payout occurs, the contract calculates the Secondary's share using the ratio: `SecondaryEquity / TotalSlotBalance`.

The `contributeAsJointMember` function contains a logic error where contributions made by the Secondary Contributor update their equity (`s_contributionShares`) but fail to update the total slot balance (`s_contributions` of the Primary). This causes the `TotalSlotBalance` denominator to be artificially low during payout calculations. Consequently, the Secondary Contributor receives an inflated share of the payout - potentially exceeding their entitlement - while the Primary Contributor receives significantly less than owed (or nothing at all).

The issue is located in the `contributeAsJointMember` function. The code handles contributions separately based on the sender:

```
if (primaryContributor == msg.sender) {
    s_contributions[groupId][primaryContributor] += msg.value;
} else {
    s_contributionShares[groupId][secondaryContributor] += msg.value;
}
```

The system relies on the invariant that `s_contributions[primary]` equals the sum of both users' deposits for that slot. By failing to add the Secondary's contribution to `s_contributions[primary]`, this invariant is broken. When `distributeJointPayout` executes:

```
uint256 totalContributions = s_contributions[groupId][primaryContributor];
uint256 secondaryShare = (totalAmount * secondaryContribution) / totalContrib
utions;
```

`totalContributions` is understated. For example, if both users contribute `0.5` in a new cycle:

- `totalContributions` increases by `0` (missing the secondary's 0.5).
- `secondaryContribution` increases by `0.5`.

The ratio `secondaryContribution / totalContributions` becomes erroneously high. In extreme cases (e.g., if the primary hasn't contributed yet this cycle

or earlier records were cleared), `secondaryContribution` could equal or exceed `totalContributions`, awarding the Secondary 100% of the slot's payout.

This can lead to:

- **Theft of Funds**: Secondary contributors systematically drain funds from Primary contributors due to the inflated payout ratio.
- **Payout DoS**: If `secondaryShare` is calculated to be greater than `totalAmount` (which is initialized as `primaryShare`), the subtraction `primaryShare -= secondaryShare` will underflow and revert, permanently locking the payout for that group.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

## Classification

**Impact Rate:** 4/5

**Likelihood Rate:** 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** High

## Recommendations

**Remediation:** Update `contributeAsJointMember` to ensure that all contributions - regardless of the sender - increment the Primary Contributor's `s_contributions` mapping to reflect the true total value of the slot.

**Resolution:** Fixed in **1829f31**. The `contributeAsJointMember()` was removed from the codebase making the current finding no longer applicable.

## Evidences

### PoC

**Reproduce:**

- **Cycle 1**: Group setup and initial payout (Admin wins).
- **Cycle 2 Contributions**:
  - Admin, User3, User4 contribute normally.

    - **Primary (User1)** contributes 0.5 ETH. s_contributions increases by 0.5.
    - **Secondary (User2)** contributes 0.5 ETH via contributeAsJointMember.
- **The Bug**: s_contributionShares increases, but s_contributions (Primary's total) does **NOT**.
- **Analysis**: TotalContributions (Primary) ends up as 1.0 ETH (0.5 from Cycle 1 accept + 0.5 from Cycle 2 primary).
    - SecondaryShareVal ends up as 1.0 ETH (0.5 from Cycle 1 accept + 0.5 from Cycle 2 secondary).
    - Ratio = 1.0 / 1.0 = 100%.
- **Payout**: Secondary gets 100% of the pot (4 ETH). Primary gets 0.

PoC Code:

```
describe("Exploit: Secondary Contributions Not Recorded (Subsequent Cycles)",
async () => {
    it("Should inflate secondary share in cycle 2 due to missing contribution
record", async () => {
    const halfShare = perShareAmount.div(2);

    await komiti.connect(user1).joinGroupWithJointContributor(groupId, user2.
address, {
    value: halfShare
    });

    await komiti.connect(user2).acceptInviteForJointContributor(groupId, user
1.address, {
    value: halfShare
    });

    await komiti.connect(user3).joinGroup(groupId, { value: perShareAmount })
;
    await komiti.connect(user4).joinGroup(groupId, { value: perShareAmount })
;

    // 2. Start the group
    await increaseTimeInSec(joinDuration + 1);
    await komiti.connect(admin).startKomiti(groupId);
    await komiti.connect(admin).setPayoutPositions(groupId, [admin.address, u
ser1.address, user3.address, user4.address]);

    // CYCLE 1 Payout
    await komiti.connect(admin).distributeFunds(groupId);

    // CYCLE 2

    // Admin
    await komiti.connect(manager).assignUserRoles([admin.address]);
```

```
    await komiti.connect(admin).contribute(groupId, { value: perShareAmount }
);

    // Others
    await komiti.connect(user3).contribute(groupId, { value: perShareAmount }
);
    await komiti.connect(user4).contribute(groupId, { value: perShareAmount }
);

    // JOINT SLOT CONTRIBUTIONS
    await komiti.connect(user1).contributeAsJointMember(groupId, user1.addres
s, user2.address, { value: halfShare });

    await komiti.connect(user2).contributeAsJointMember(groupId, user1.addres
s, user2.address, { value: halfShare });

    // 3. Analyze State Before Payout
    const user1BalanceBefore = await user1.getBalance();
    const user2BalanceBefore = await user2.getBalance();

    // 4. Distribute Cycle 2 Payout (to User1/Joint)
    await komiti.connect(admin).distributeFunds(groupId);

    const user1BalanceAfter = await user1.getBalance();
    const user2BalanceAfter = await user2.getBalance();

    // 5. Analyze Results
    const payoutAmount = perShareAmount.mul(4); // 4 ETH
    const user1Received = user1BalanceAfter.sub(user1BalanceBefore);
    const user2Received = user2BalanceAfter.sub(user2BalanceBefore);

    expect(user2Received).to.be.closeTo(payoutAmount, utils.parseEther("0.01"
));
    expect(user1Received).to.be.closeTo(BigNumber.from(0), utils.parseEther("
0.01"));
    });
});
```

**Results:**

```
Exploit: Secondary Contributions Not Recorded (Subsequent Cycles)
      ✔ Should inflate secondary share in cycle 2 due to missing contribution
record
```

## F-2025-14229 - Missing USER_ROLE Check Allows Unauthorized Participants in Joint Groups And Payouts - Medium

**Description:**

The function `joinGroupWithJointContributor()` allows a user to join a group with a secondary contributor but lacks a check to verify that the secondary contributor has been granted the required `USER_ROLE`. This allows registration of secondary contributors who do not have the necessary permissions to participate in the system, potentially causing permission inconsistencies and unexpected behavior.

```
  function joinGroupWithJointContributor(uint256 groupId, address secondaryCon
tributor)
        external
        payable
        whenNotPaused
        checkIfGroupExit(groupId)
        onlyRole(USER_ROLE)
    {
        require(secondaryContributor != address(0), "Komiti: Zero Address not
allowed");
        Group storage group = s_groups[groupId];

        require(!group.isPayoutStarted, "Komiti: Cannot join after payout sta
rts");
        require(block.timestamp < group.joinDeadline, "Komiti: Cannot join af
ter deadline");
        require(s_contributions[groupId][msg.sender] == 0, "Komiti: User alre
ady joined");
        require(msg.value >= group.perShareAmount / 2, "Komiti: Incorrect amo
unt sent"); // Half for this user and half for secondary contributor

        require(
            !s_isJointContributor[groupId][secondaryContributor], "Komiti: Se
condary contributor already registered"
        );

        s_jointContributorPrimary[groupId][secondaryContributor] = msg.sender
;
        s_isJointContributor[groupId][secondaryContributor] = true;

        s_jointContributorsDetails[groupId][msg.sender].primaryContributor =
msg.sender;
        s_contributionShares[groupId][secondaryContributor] = 0;

        emit Komiti__JointContributorAdded(groupId, msg.sender, secondaryCont
```

```
ributor);
        emit Komiti__UserJoined(groupId, msg.sender);
    }
```

Although the `acceptInviteForJointContributor()` function enforces that the secondary contributor must have the `USER_ROLE`, this validation occurs **after** the secondary contributor has already been registered in `joinGroupWithJointContributor()`. Moreover, to reregister the secondary contributor, the primary contributor will have to pay the half of the `perShareAmount` again. This sequencing is problematic because it allows unverified addresses to be recorded as secondary contributors before their privileges are confirmed. Registering secondary contributors without prior role verification undermines the integrity of access control, as other contract logic assumes all secondary contributors hold the necessary `USER_ROLE`. In its current form, the contract permits "unprivileged" addresses to be assigned as secondary contributors, violating core assumptions about role-based participation and potentially weakening the system's security model.

Another instance of missing permissions validation was found in the `setPayoutPositions()` function where the `ADMIN_ROLE` can add users to the `s_payoutPositions` without validating that these users have the `USER_ROLE` role or have previously been added to the group as members.

```
function setPayoutPositions(uint256 groupId, address[] calldata users) extern
al checkIfGroupExit(groupId) onlyRole(ADMIN_ROLE) {
        Group storage selectedGroup = s_groups[groupId];
        uint256 noOfGroupMembers = selectedGroup.members.length;
        ...
        } else {
            // Assuming the admin will send the correct address that have joi
ned the group
            for (uint256 i = 0; i < users.length; i++) {
                s_payoutPositions[groupId][users[i]] = i;
                emit Komiti__PayoutPositionAssigned(groupId, users[i], i);
            }
        }
    }
```

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

## Classification

| | |
|---|---|
| **Impact Rate:** | 2/5 |
| **Likelihood Rate:** | 5/5 |
| **Exploitability:** | Semi-Dependent |
| **Complexity:** | Simple |
| **Severity:** | Medium |

## Recommendations

**Remediation:** Add an explicit role check at the start of `joinGroupWithJointContributor()` to ensure the secondary contributor has been granted a `USER_ROLE`:

```
require(hasRole(USER_ROLE, secondaryContributor), "Komiti: Secondary contributor must have USER_ROLE");
```

In `setPayoutPositions()` function In addition to the `USER_ROLE` validation, consider checking that the users in the users array have previously been added to the current group.

**Resolution:** Fixed in `1829f31`. The codebase no longer contains the `joinGroupWithJointContributor()` functionality. In addition to that, the following `USER_ROLE` check was implemented in the `setPayoutPositions()` function:

```
...
if (!hasRole(USER_ROLE, users[i])) revert UserHasNotJoined();
...
```

## F-2025-14233 - Broken Participant Invariant Due to Unrestricted USER_ROLE Revocation - Medium

**Description:**

The protocol documentation establishes a strict invariant:

**"Ensure all participants have USER_ROLE."**

However, the `revokeUserRoles()` function allows the `MANAGER_ROLE` to revoke `USER_ROLE` **even from users who have already joined a group and contributed funds**. This directly violates the protocol's role consistency requirements and creates multiple broken states where a participant is simultaneously part of a group but not allowed to perform user operations.

This leads to blocked contributions, blocked withdrawals, broken joint-contributor flows, and inability to continue participating—effectively bricking the user inside the system.

**Impact:**

- Permanent loss of ability to contribute, preventing payouts from progressing.
- Deadlocked joint-contributor states, where secondary contributors can't finalize or unwind.
- Permanent fund lock for users who cannot complete the required functions.
- Groups enter an inconsistent or corrupted state, violating invariants relied upon throughout the protocol.
- Ability for a revoked user to be selected as the winner.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Accepted

### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 4/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Medium

### Recommendations

**Remediation:** It is recommended to:

- Prevent revoking `USER_ROLE` from active group participants in the `revokeUserRoles()` function. Additionally, override `AccessControl::revokeRole()` and `AccessControl::renounceRole()` to prevent to revoke roles via default AccessControl functionality.
- Moreover, introduce a separate function where revoked users could recover their contributions.

**Resolution:** `Accepted`. The client is aware of the finding and accepted the risk.

## F-2025-14253 - Missing Refund Mechanism for Regular Members Leads To Stuck Assets - Medium

**Description:** The smart-contract provides a `returnFunds()` function, but it only applies to joint-contributor workflows, allowing a primary contributor to unwind an invitation before payout starts.

```
 function returnFunds(uint256 groupId, address secondaryContributor)
        external
        payable
        checkIfGroupExit(groupId)
        onlyRole(USER_ROLE)
    {
        Group storage group = s_groups[groupId];

        require(!group.isPayoutStarted, "Komiti: Cannot return funds after pa
yout starts");
        require(
            s_jointContributorPrimary[groupId][secondaryContributor] == msg.s
ender,
            "Komiti: Only primary contributor can return amount"
        );

        require(s_contributionShares[groupId][secondaryContributor] == 0, "Ko
miti: Invite already accepted");

        s_jointContributorPrimary[groupId][secondaryContributor] = address(0)
;
        s_isJointContributor[groupId][secondaryContributor] = false;

        address[] storage secondaryContributors = s_jointContributorsDetails[
groupId][msg.sender].secondaryContributors;

        for (uint256 i = 0; i < secondaryContributors.length; i++) {
            if (secondaryContributors[i] == secondaryContributor) {
                secondaryContributors[i] = secondaryContributors[secondaryCon
tributors.length - 1];
                secondaryContributors.pop();
                break;
            }
        }

        delete s_jointContributorsDetails[groupId][msg.sender];

        uint256 amountToReturn = s_contributions[groupId][msg.sender];
```

```
        s_contributions[groupId][msg.sender] = 0;

        (bool success,) = msg.sender.call{value: amountToReturn}("");
        require(success, "Komiti: Transfer failed");

        emit Komiti__JointContributorInviteReturned(groupId, msg.sender, seco
ndaryContributor);
    }
```

However, regular users who joined via `joinGroup()` have no mechanism to reclaim their funds if the `Komiti` does not start, the group overfills, stalls, or is abandoned. This contradicts expected user safety guarantees and creates a scenario where funds contributed by standard participants become **permanently locked**, even though the payout process never begins.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

## Classification

**Impact Rate:** 5/5

**Likelihood Rate:** 4/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Medium

## Recommendations

**Remediation:** It is recommended to:

- implement a refund path for regular users before payouts begin.
- ensure that the refund path operate normally both for joint and non-joint participants.
- add admin emergency refund capabilities for abandoned or corrupted groups.
- consider covering the aforementioned system flows with the tests to ensure that the users receive their contributions back.

**Resolution:** In commit `1829f31`, the public `refundMember` function that allows any contributing member to withdraw their funds before the payout cycle starts

is introduced, resolving the issue where regular users had no mechanism to reclaim funds from stalled or inactive groups.

## F-2025-14273 - Excess Contributions Become Permanently Locked Due to Non-Exact Deposit Enforcement - Medium

**Description:**

The `Komiti::joinGroup()`, `Komiti::joinGroupWithJointContributor()`, `Komiti::acceptInviteForJointContributor()`, and `Komiti::contribute()` functions all validate user deposits using a greater-than-or-equal check:

```
require(msg.value >= group.perShareAmount, "Komiti: Incorrect amount sent");
```

This design allows users to send more funds than required by the protocol.

However, the payout mechanism in `Komiti::distributeFunds()` only uses:

```
uint256 payoutAmount = (selectedGroup.perShareAmount * selectedGroup.members.length);
```

This means:

- Only the minimum required contribution is considered for distribution.
- Any extra amount contributed above `perShareAmount` is not refunded, not redistributed, and not withdrawable.
- These surplus funds accumulate inside the contract without any mechanism for recovery.

As a result, all excess contributions become permanently locked, forming an unrecoverable ether sink inside the protocol.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 5/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Medium

### Recommendations

**Remediation:** It is recommended to:

- enforce exact contribution amounts: replace all >= checks with exact equality

```
require(msg.value == group.perShareAmount, "Komiti: Exact share amount requir
ed");
```

or

```
require(msg.value == group.perShareAmount / 2, "Komiti: Exact half-share amou
nt required");
```

This enforces protocol consistency and eliminates accidental overpayments.

- alternatively, consider redesign the distribution mechanism and ensure that all user contributions are distributed.
- otherwise, introduce an admin refund mechanism to ensure that all leftovers that are left in the contract after withdrawing the `perShareAmount*amount_of_group_members` can be safely withdrawn.

**Resolution:** Fixed in `1829f31`. Now in order to join the group or to contribute via `joinGroup()` and `contribute()` functions, the user needs to pay the exact `perShareAmount` as follows:

```
...
if (msg.value != group.perShareAmount) revert InvalidContributionAmount();
...
```

## F-2025-14287 - DoS via Concurrent Joint Contributor Invites - Medium

**Description:**

The `Komiti` contract allows a Primary Contributor to invite a Secondary Contributor to share a slot via `joinGroupWithJointContributor`. The contract intends to limit each Primary Contributor to a single slot, enforcing this by checking `s_contributions[groupId][msg.sender] == 0`.

An issue allows a Primary Contributor to bypass the single-slot restriction by sending multiple invites concurrently before any are accepted. Because `s_contributions` is only updated when a Secondary accepts an invite (in `acceptInviteForJointContributor`), the check in `joinGroupWithJointContributor` passes multiple times. This results in the Primary Contributor being added to the `group.members` array multiple times, creating "duplicate slots." Since a user can only be paid once per group cycle, these duplicate slots disrupt the payout logic, causing the final payout cycle to fail (DoS) and permanently locking the funds collected for that cycle.

The issue stems from the timing gap between sending an invite and its acceptance.

- **Concurrent Invites**: The Primary calls `joinGroupWithJointContributor` twice (or more) in rapid succession with different Secondaries.

```
    require(s_contributions[groupId][msg.sender] == 0, "Komiti: User already joined");
```

This check passes for both calls because `s_contributions` remains `0` until an invite is accepted.

- **Multiple Acceptances**: When the Secondaries accept via `acceptInviteForJointContributor`, each transaction executes:

```
    group.members.push(primaryContributor);
```

This pushes the Primary's address to the members array multiple times.

- **Payout Logic Failure**: In `distributeFunds`, the contract iterates through members.
  - When the Primary wins the first time, `s_hasReceivedPayout[groupId][primary]` is set to true.
  - When the loop encounters the Primary's duplicate slot in a later cycle, it skips them because they have already been paid (`!s_hasReceivedPayout` is `false`).
  - The cycle proceeds to pay the next unique member early.

- **Final Cycle**: When the group reaches the final intended cycle, all unique members have already been paid. The loop completes without finding an eligible winner (`winner == address(0)`), and the transaction reverts:

```
require(winner != address(0), "Komiti: No eligible winner found");
```

The funds collected for the final cycle are permanently locked in the contract. This can lead to:

- The final cycle of the group cannot be processed.
- The entire pot for the final cycle (which has been collected from all members) is irretrievably stuck.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

## Classification

**Impact Rate:** 4/5

**Likelihood Rate:** 3/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Medium

## Recommendations

**Remediation:** Update the Primary Contributor's `s_contributions` mapping immediately when `joinGroupWithJointContributor` is called. This prevents the `s_contributions == 0` check from passing on subsequent calls, effectively blocking concurrent invites.

**Resolution:** Fixed in `1829f31`. The functions `acceptInviteForJointContributor()` and `joinGroupWithJointContributor()` have been removed from the codebase.

## Evidences

**PoC**

**Reproduce:**

1. **Concurrent Invites**: User1 invites User2, then immediately invites User3. s_contributions is 0 for both calls.
2. **Acceptance**: User2 and User3 both accept. Each pushes User1 to group.members.
3. **Result**: members array contains [Admin, User1, User1].
4. **Payout Logic**:

- Cycle 2: User1 (Instance 1) gets paid.
- Cycle 3: User1 (Instance 2) is skipped because s_hasReceivedPayout[User1] is true. The payout goes to the *next* member (User4).
- Cycle 4: Everyone unique has been paid. No winner found. DoS.

PoC Code:

```
it("Should allow primary to occupy multiple slots via concurrent invites", as
ync () => {
    const halfShare = perShareAmount.div(2);

    // 2. User1 sends TWO invites BEFORE anyone accepts.
    // Invite User2
    await komiti.connect(user1).joinGroupWithJointContributor(groupId, user2.
address, { value: halfShare });

    // Invite User3
    await komiti.connect(user1).joinGroupWithJointContributor(groupId, user3.
address, { value: halfShare });

    // 3. Both Accept
    await komiti.connect(user2).acceptInviteForJointContributor(groupId, user
1.address, { value: halfShare });
    await komiti.connect(user3).acceptInviteForJointContributor(groupId, user
1.address, { value: halfShare });

    // 4. Fill last slot (Slot 4)
    await komiti.connect(user4).joinGroup(groupId, { value: perShareAmount })
;

    // 5. Start Komiti
    await increaseTimeInSec(joinDuration + 1);
    await komiti.connect(admin).startKomiti(groupId);

    await komiti.connect(admin).setPayoutPositions(groupId, [admin.address, u
ser1.address, user1.address, user4.address]);

    // CYCLE 1
    await komiti.connect(admin).distributeFunds(groupId); // Admin paid.

    // CYCLE 2
```

```
    await komiti.connect(manager).assignUserRoles([admin.address]);
    await komiti.connect(admin).contribute(groupId, { value: perShareAmount }
);
    await komiti.connect(user1).contributeAsJointMember(groupId, user1.addres
s, user2.address, { value: halfShare });
    await komiti.connect(user2).contributeAsJointMember(groupId, user1.addres
s, user2.address, { value: halfShare });
    await komiti.connect(user1).contributeAsJointMember(groupId, user1.addres
s, user3.address, { value: halfShare });
    await komiti.connect(user3).contributeAsJointMember(groupId, user1.addres
s, user3.address, { value: halfShare });
    await komiti.connect(user4).contribute(groupId, { value: perShareAmount }
);

    // Payout to User1
    await komiti.connect(admin).distributeFunds(groupId);

    // CYCLE 3
    await komiti.connect(admin).contribute(groupId, { value: perShareAmount }
);
    await komiti.connect(user4).contribute(groupId, { value: perShareAmount }
);
    await komiti.connect(user4).contribute(groupId, { value: perShareAmount }
);
    await komiti.connect(user4).contribute(groupId, { value: perShareAmount }
);

    await komiti.connect(admin).distributeFunds(groupId);

    // Verify User4 got paid early
    const info = await komiti.getMemberInfoByGroup(groupId, user4.address);
    expect(info.hasReceivedPayout).to.be.true;

    // CYCLE 4:
    await komiti.connect(admin).contribute(groupId, { value: perShareAmount }
);

    await expect(
    komiti.connect(admin).distributeFunds(groupId)
    ).to.be.revertedWith("Komiti: No eligible winner found");
});
```

**Results:**

```
Exploit: DoS via Multiple Slots for Same Primary (Duplicate Members)
      ✔ Should allow primary to occupy multiple slots via concurrent invites
```

## F-2025-14224 - Weak RNG: block.timestamp Enables Predictable and Manipulable Payout Shuffling - Low

**Description:**

The randomized payout selection mechanism is used inside the `setPayoutPositions()` function to calculate the payout positions necessary for the payouts distribution process:

```
..
if (selectedGroup.isRandomized) {
            uint256[] memory positions = new uint256[](noOfGroupMembers);
            for (uint256 i = 0; i < noOfGroupMembers; i++) {
                positions[i] = i;
            }

            for (uint256 i = noOfGroupMembers - 1; i > 0; i--) {
                uint256 j = uint256(keccak256(abi.encodePacked(block.timestam
p, i, msg.sender))) % (i + 1);

                (positions[i], positions[j]) = (positions[j], positions[i]);

            }

            for (uint256 i = 0; i < noOfGroupMembers; i++) {
                s_payoutPositions[groupId][selectedGroup.members[i]] = positi
ons[i];
                emit Komiti__PayoutPositionAssigned(groupId, selectedGroup.me
mbers[i], positions[i]);
            }
..
```

The use of `keccak256` hash functions on predictable values like `block.timestamp`, `block.number`, or similar data, including modulo operations on these values, should be avoided for generating randomness, as they are easily predictable and manipulable. Miners can shift timestamps within a ~15 second window, enabling them to influence shuffle outcomes.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Accepted

### Classification

**Impact Rate:** 2/5

| | |
|---|---|
| **Likelihood Rate:** | 3/5 |
| **Exploitability:** | Semi-Dependent |
| **Complexity:** | Simple |
| **Severity:** | Low |

### Recommendations

**Remediation:** It is recommended to consider using another source of randomness such as Chainlink VRF. Chainlink VRF provides a provably fair and verifiable source of randomness.

**Resolution:** `Accepted`. The client is aware of the finding and accepted the risk.

## F-2025-14226 - Missing startTime Validation Enables Creation of Invalid Groups That Fail Existence Checks - Low

**Description:**

The `Komiti::createGroup()` function fails to validate the `startTime` parameter, allowing an `ADMIN_ROLE` or `ORGANIZER_ROLE` to create a group whose `startTime` is set to 0 that makes the group logically invalid. Because other parts of the protocol implicitly assume that `startTime` is a meaningful, non-zero scheduling value, this omission enables the creation of **non-existent or permanently invalid groups** that break lifecycle assumptions and may never proceed into the payout phase.

function `createGroup()`

```
    function createGroup(
        bool isRandomized,
        uint256 perShareAmount,
        uint256 totalSlots,
        uint256 startTime,
        uint256 cycleDuration,
        uint256 joinDuration
    ) external payable {
        require(perShareAmount > 0, "Komiti: Invalid share amount");
        require(totalSlots > 1, "Komiti: Total slots must be more than one");
        require(cycleDuration == 7 days || cycleDuration == 30 days, "Komiti:
Invalid cycle duration");
        require(
            hasRole(ADMIN_ROLE, msg.sender) || hasRole(ORGANIZER_ROLE, msg.se
nder),
            "Komiti: Only Admin or Organizer can create group"
        );
        require(msg.value == perShareAmount, "Komiti: Admin must pay first sh
are");

        s_groupCounter += 1;
        Group storage newGroup = s_groups[s_groupCounter];

        newGroup.isRandomized = isRandomized;
        newGroup.isPayoutStarted = false;
        newGroup.startTime = startTime;
        newGroup.totalSlots = totalSlots;
        newGroup.perShareAmount = perShareAmount;
        newGroup.cycleDuration = cycleDuration;
        newGroup.currentPayoutIndex = 0;
        newGroup.joinDeadline = block.timestamp + joinDuration;
        newGroup.organizer = msg.sender;
```

```
        newGroup.members.push(msg.sender);
        s_contributions[s_groupCounter][msg.sender] = perShareAmount;

        emit Komiti__GroupCreated(s_groupCounter, isRandomized, perShareAmoun
t, totalSlots);
    }
```

The modifier `checkIfGroupExit` invalidates all groups where the `startTime` is set to 0.

```
  modifier checkIfGroupExit(uint256 groupId) { // @audit-info typo in the nami
ng
        Group storage group = s_groups[groupId];
        require(group.startTime != 0, "Komiti: group does not exist");
        _;
    }
```

The lack of `startTime` validation leads to several functional failures:

- **Creation of unusable groups.** Any group created with startTime = 0 (or another invalid timestamp) will always fail `checkIfGroupExit()`, preventing:
  - joining the group,
  - updating group parameters,
  - starting payouts,
  - retrieving group details,
  - distributing funds.
- **Permanent protocol inconsistencies.** Unusable groups still increment `s_groupCounter`, pollute storage, and appear in `getTotalGroups()`.
- **Unexpected UI / backend failures.** Off-chain systems or front-ends will detect the group via events but fail all subsequent interactions.
- **Broken lifecycle invariants.** The protocol implicitly assumes that a created group **always** has a valid, non-zero start time. This invariant is violated, allowing logic flows that lead to unexpected revert paths.
- **Loss of admin payment during createGroup().** The admin or organizer is required to send the first share via `msg.value == perShareAmount`. When the group is created with startTime = 0, the group becomes non-existent under protocol invariants and can never proceed to payout, contribution cycles, or refunds. As a result, the admin's initial deposit becomes locked inside an invalid group, creating a direct financial loss.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 2/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Low

### Recommendations

**Remediation:** It is recommended to add strict validation for the `startTime` parameter in `createGroup()`:

```
require(startTime != 0, "Komiti: Invalid start time");
require(startTime >= block.timestamp, "Komiti: startTime must be now or in th
e future");
```

**Resolution:** Fixed in `1829f31`. The `startTime` value of a group is now validated in the `createGroup()` function and must always be greater than zero.

## F-2025-14230 - Missing Primary Contributor Validation Enables Uninvited Users to Join Groups - Low

**Description:**

The `acceptInviteForJointContributor()` function incorrectly allows any address with `USER_ROLE` to join a group as a secondary contributor **without ever being invited or registered** through the `joinGroupWithJointContributor()` function.

This occurs because the function does not validate that the `primaryContributor` argument is a **non-zero,** valid primary contributor, allowing a user to bypass invite registration by directly calling:

```
acceptInviteForJointContributor(groupId, address(0))
```

```
function acceptInviteForJointContributor(uint256 groupId, address primaryCont
ributor)
        external
        payable
        whenNotPaused
        checkIfGroupExit(groupId)
        onlyRole(USER_ROLE)
    {
        Group storage group = s_groups[groupId];

        require(!group.isPayoutStarted, "Komiti: Cannot join after payout sta
rts");
        require(block.timestamp < group.joinDeadline, "Komiti: Cannot join af
ter deadline");
        require(
            s_jointContributorPrimary[groupId][msg.sender] == primaryContribu
tor, "Komiti: Invalid primary contributor"
        );
        require(s_contributionShares[groupId][msg.sender] == 0, "Komiti: User
already accepted Invite");
        require(msg.value >= group.perShareAmount / 2, "Komiti: Incorrect amo
unt sent"); // Half for this user and half for secondary contributor

        group.members.push(primaryContributor);
        s_isJointContributor[groupId][primaryContributor] = true;
        s_contributionShares[groupId][msg.sender] = msg.value;
        s_jointContributorsDetails[groupId][primaryContributor].secondaryCont
ributors.push(msg.sender);
        s_contributions[groupId][primaryContributor] += msg.value;
        emit Komiti__JointContributorInviteAccepted(groupId, primaryContribut
```

```
or, msg.sender);
        emit Komiti__UserJoined(groupId, msg.sender);
    }
```

If `s_jointContributorPrimary[groupId][msg.sender]` is also `0x0` (the default value for all unmapped entries), the check passes, and the user is added to the group even though:

- They were **never** invited by any primary contributor
- They were **never** registered as a joint contributor
- They completely bypass the intended joint-contributor flow

**Impact**:

- Contribution funds being attributed to address(0). The system records msg.value as a contribution for a non-existent participant (0x0). This breaks multiple assumptions in the protocol:
  - address(0) never joined the group
  - address(0) never paid the required initial share
  - address(0) is not included in members[]
  - address(0) has no role and no permissions

Yet the contract treats the zero address as if it were a valid contributing member.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

## Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 4/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Low

## Recommendations

**Remediation:** Add a strict validation at the start of `acceptInviteForJointContributor()`:

```
require(primaryContributor != address(0), "Komiti: Invalid primary contributo
r");
```

**Resolution:** Fixed in `1829f31`. The finding was fixed due to the absence of the `acceptInviteForJointContributor()` functionality in the latest version of the codebase.

## F-2025-14267 - Unset Payout Positions Default to Index 0, Allowing Any Unpaid Member to Illegitimately Win First Payout - Low

**Description:**

The payout mechanism relies on the `s_payoutPositions` mapping to determine which participant should receive the payout for the current round. It can be seen in the `distributeFunds()` function below:

```
...
for (uint256 i = 0; i < totalMembers; i++) {
     address member = selectedGroup.members[i];
     if (
        !s_hasReceivedPayout[groupId][member]
          && (winner == address(0) || s_payoutPositions[groupId][member] == s
electedGroup.currentPayoutIndex)) {
                winner = member;
                break;
            }
 }
...
```

However, if a user **does not exist** in the payout positions mapping, Solidity returns the default value 0. Because the first payout round (`currentPayoutIndex`) also equals 0, a first unpaid user with no assigned position is mistakenly treated as a valid candidate for payout index 0. This results in unselected users receiving the first payout.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Accepted

### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 3/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Low

### Recommendations

**Remediation:** It is recommended to redesign the current winner selection process:

- consider avoiding a zero payout position in `s_payoutPositions` mapping.
- redefine the process of setting the `currentPayoutIndex` value and prevent it from being zero.
- test the behavior of the selection mechanism, whether the unselected user can be chosen as the winner.

**Resolution:** `Accepted`, the client has ackowledged the finding.

## F-2025-14285 - Protocol Insolvency Due to Premature Payout Triggering - Low

**Description:**

The `Komiti` contract operates as a Rotating Savings and Credit Association (ROSCA). In each cycle, all group members are expected to contribute a fixed `perShareAmount`. Once collected, the `distributeFunds` function is called by the `Admin` to pay out the total pot (calculated as `perShareAmount * totalMembers`) to the cycle's designated winner.

The `distributeFunds` function calculates the payout amount based on the total number of members in the group, assuming all members have contributed for the current cycle. However, it fails to verify if the contract actually holds sufficient funds (i.e., if all members have actually paid). If an `Admin` triggers the payout before collecting all contributions, the contract attempts to transfer more funds than it holds, causing the transaction to revert. This leads to a denial of service where the payout process is blocked until the missing funds are supplied.

In `distributeFunds`, the payout amount is determined by the group size:

```
uint256 payoutAmount = (selectedGroup.perShareAmount * selectedGroup.members.
length);


if (/*...*/) {
    …
} else {
    payable(winner).transfer(payoutAmount);
}
```

The contract logic does not check `address(this).balance` or verify that every member has called contribute for the current `currentPayoutIndex` cycle. If a group has 4 members (requiring a 4 units payout) but only 3 members have contributed (3 units balance), the calculation `4 * 1 unit = 4 unit` remains unchanged. When `transfer` is executed, the VM reverts the transaction due to insufficient balance. This creates an operational hazard where a single non-paying member (or Admin negligence) can "brick" the cycle's payout transaction. While the funds are not lost, the protocol halts, requiring manual intervention (e.g., someone else covering the cost) to proceed.

This makes the payout functionality unusable for the current cycle if the pot is not fully funded.

**Assets:**

- contracts/RYTStablecoin.sol [ryt-stablecoin-develop.zip]

**Status:** Fixed

## Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 4/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Low

## Recommendations

**Remediation:** Add a check at the start of `distributeFunds` to ensure `address(this).balance >= payoutAmount`.

**Resolution:** Fixed in `1829f31`. The following validation was added to the `distributeFunds()` function:

```
...
uint256 payoutAmount = (selectedGroup.perShareAmount * totalMembers);
if (address(this).balance < payoutAmount) revert InsufficientContractBalance(
);
...
```

## Evidences

### PoC

**Reproduce:**

- **Cyle 1:** Full group (Admin + 3 Users), full pot (4 ETH), Payout empties the pot.
- **Cycle 2**: Admin, User1, and User2 contribute (3 ETH total).
- **User3 fails to contribute**.
  - Contract Balance = 3 ETH.
- **Exploit**: Admin calls distributeFunds.
  - Contract calculates payoutAmount = 4 Members * 1 ETH = 4 ETH.
  - Contract attempts to transfer 4 ETH.

PoC Code:

```
it("Should revert distribution if admin triggers payout before all members co
ntribute", async () => {
```

```
    // 1. Setup Group
    await komiti.connect(user1).joinGroup(groupId, { value: perShareAmount })
;
    await komiti.connect(user2).joinGroup(groupId, { value: perShareAmount })
;
    await komiti.connect(user3).joinGroup(groupId, { value: perShareAmount })
;

    // 2. Start Group
    await increaseTimeInSec(joinDuration + 1);
    await komiti.connect(admin).startKomiti(groupId);
    await komiti.connect(admin).setPayoutPositions(groupId, [admin.address, u
ser1.address, user2.address, user3.address]);

    await komiti.connect(admin).distributeFunds(groupId);

    const contractBalanceAfterCycle1 = await ethers.provider.getBalance(komit
i.address);
    expect(contractBalanceAfterCycle1).to.equal(0);

    await komiti.connect(manager).assignUserRoles([admin.address]); // Fix fo
r admin role
    await komiti.connect(admin).contribute(groupId, { value: perShareAmount }
);
    await komiti.connect(user1).contribute(groupId, { value: perShareAmount }
);
    await komiti.connect(user2).contribute(groupId, { value: perShareAmount }
);

    await expect(
    komiti.connect(admin).distributeFunds(groupId)
    ).to.be.reverted;
});
```

**Results:**

```
  Exploit: Insolvency Risk (Premature Payout)
      ✔ Should revert distribution if admin triggers payout before all member
s contribute
```

## F-2025-14286 - Missing Minimum Slot Validation in updateKomiti() Allows Bypassing Group Size Invariant - Low

**Description:**

The protocol correctly enforces a minimum group size constraint during group creation. In `createGroup()`, the contract requires:

```
...
require(totalSlots > 1, "Komiti: Total slots must be more than one");
...
```

This invariant ensures that a Komiti group must have at least **two participants**, which is essential for a rotating payout model.

However, the update function - `updateKomiti()` - does not enforce the same constraint:

```
...
if (totalSlots != 0) {
    selectedGroup.totalSlots = totalSlots;
}
...
```

Because no validation is performed here, any user with permission (`ADMIN_ROLE` or organizer) can update the group and set: `totalSlots` = 1. This bypasses the intended invariant from `createGroup()`, breaks payout logic assumptions, and can result in an invalid or permanently stuck group.

**Status:** Fixed

### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 2/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Low

### Recommendations

**Remediation:** It is recommended to add validation in `updateKomiti()` to enforce the same minimum requirements as in `createGroup()`:

```
if (totalSlots != 0) {
    require(totalSlots > 1, "Komiti: Total slots must be more than one");
    selectedGroup.totalSlots = totalSlots;
}
```

Also ensure that new `totalSlots` > current number of joined members.

This preserves protocol invariants and prevents disabling or breaking running groups.

**Resolution:** Fixed in `1829f31`. The following validation checks were introduced in the `updateKomiti()` function to enforce the system requirements:

```
...
if (totalSlots <= 1) revert InvalidTotalSlots();
if (totalSlots < selectedGroup.members.length) revert InvalidTotalSlots();
...
```

## F-2025-14306 - Refund Logic Becomes Irreversible After Invitation Acceptance, Leading to Fund Lock for Both Joint Contributors - Low

**Description:**

The `Komiti` contract implements a limited refund mechanism for joint contributions, but only covers a single, narrow scenario. Currently, the protocol allows a **primary contributor** to refund their contribution **only if** the secondary contributor has **not yet accepted** the invitation via `acceptInviteForJointContributor()`.

Once the secondary contributor accepts the invitation and deposits their funds:

- The **primary contributor** can no longer refund their contribution
- The **secondary contributor** has no refund mechanism at all
- No refund is possible even if:
  - the `Komiti` has not started
  - no payout has been executed
  - the group is misconfigured
  - the group never reaches a valid operational state

In other words, invitation acceptance permanently locks funds for both parties, regardless of the future state of the group. This behavior is not explicitly documented and contradicts user expectations that funds should remain recoverable before the payout phase begins.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 3/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Low

### Recommendations

**Remediation:** It is recommended to review and potentially redesign the `Komiti` refund mechanism by introducing a **comprehensive refund** procedure for joint

contributors prior to `Komiti` start. For example, allow either contributor to trigger a refund **before the first payout**, refunding both parties proportionally.

**Resolution:** In commit `1829f31`, the complex joint-contribution logic has been replaced with a universal `refundMember` function, allowing any participant to withdraw their funds at any time before payouts begin, eliminating the scenario where funds could become permanently locked.

## F-2025-14219 - Presence of Debugging and Commented Code in Production Contract - Info

**Description:**

The `Komiti.sol` contract contains debugging code (`console.log` statements) and commented-out imports from the Hardhat development environment, which are not intended for use in a production code. In addition to that, the latest version of the code contains commented out snippets of code which are not meant to be used.

The `console.sol` library, used with Hardhat for development, allows developers to print output to the console for debugging purposes during local testing. However, these debugging statements should not be present in the deployed contracts as they can bloat the contract size, reduce code readability, and introduce unnecessary complexity.

The following sections of code are affected:

```
import {Pausable} from "@openzeppelin/contracts/security/Pausable.sol";
import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol
";
import {ReentrancyGuard} from "@openzeppelin/contracts/security/ReentrancyGua
rd.sol";
// import "hardhat/console.sol";
```

function `joinGroupWithJointContributor()`

```
...
// Not needed
        // group.contributions[msg.sender] += msg.value;
        /*
        1. This will be done now when invite is accepted
            group.members.push(msg.sender);
            group.jointContributorsDetails[msg.sender].secondaryContributors.
push(secondaryContributor);
        2. secondaryContributor will accept invite do verification then can c
ontribute
            require(hasRole(USER_ROLE, secondaryContributor), "Komiti: Second
ary contributor must have USER_ROLE");
        */

...
```

function `setPayoutPositions()`

```
 ...
// // Debugging
                // console.log(positions[j], positions[i]);
                // console.log("--------------------");
...
// // Debugging
            // console.log("--------------------");
            // console.log(positions[0], positions[1], positions[2], position
s[3]);
            // console.log("--------------------");
            // for (uint256 i = 0; i < noOfGroupMembers; i++) {
            //     console.log(selectedGroup.payoutPositions[selectedGroup.me
mbers[i]], selectedGroup.members[i]);
            // }
...
```

Debugging code and commented-out imports reduce code quality and readability.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

## Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

## Recommendations

**Remediation:** Remove all debugging and commented-out code from the contract to ensure a clean and production-ready codebase.

**Resolution:** Fixed in `1829f31`. The debugging and commented out code was successfully removed from the codebase.

## F-2025-14220 - Risks from Using Outdated Solidity and OpenZeppelin Libraries - Info

**Description:**

The project is currently built using Solidity version `0.8.0` and OpenZeppelin contracts version `^4.8.1.` Both these versions have documented vulnerabilities and known issues that may not affect the project's current state but pose significant risks upon future updates or if certain features are utilized.

- OpenZeppelin Vulnerabilities: Details on issues with OpenZeppelin contracts version `4.8.1` can be reviewed at Snyk.
- Solidity Issues: Known bugs in the Solidity version currently in use are documented on the Solidity Github.

These outdated components can expose the project to security risks, especially during future updates or if new features are utilized.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Accepted

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 4/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:**

Identify the latest stable versions of both Solidity and OpenZeppelin that are free from critical vulnerabilities affecting your project's requirements and goals. Consider versions that:

- Have been out long enough to be tested and vetted by the community.
- Offer fixes for any known issues that could impact your deployment.
- Support features and functionalities necessary for your project.

**Resolution:** Accepted in `1829f31`. The Solidity version was set to 0.8.20, while the Openzeppelin version was left unchanged.

## F-2025-14221 - Use of transfer() instead of call() to send native tokens - Info

**Description:**

The project utilizes the `transfer()` method for sending native currency to a recipient. However, if the recipient is a smart contract, the fixed gas limit associated with `transfer()` in Solidity might prevent the transaction from being completed.

In earlier Solidity versions, the `transfer()` function was favored for its straightforwardness and built-in reentrancy guard. Yet, it has become known for issues tied to its rigid gas ceiling of 2300.

Leveraging `transfer()` could inadvertently cause the transaction to revert if the recipient contract's `receive()` or `fallback()` methods require more than 2300 Gas for execution or if the gas price will be higher in the future.

The usage of the `transfer()` method was found in the following functions `distributeFunds()` and `distributeJointPayout()`, which can be seen in the code snippet below:

`distributeFunds()`

```
function distributeFunds(uint256 groupId) public nonReentrant checkIfGroupExi
t(groupId) onlyRole(ADMIN_ROLE) {
        Group storage selectedGroup = s_groups[groupId];

        require(selectedGroup.isPayoutStarted, "Komiti: Payout not started");
        require(selectedGroup.members.length > 0, "Komiti: No members in grou
p");

        address winner = address(0);
        uint256 totalMembers = selectedGroup.members.length;

        for (uint256 i = 0; i < totalMembers; i++) {
            address member = selectedGroup.members[i];
            if (
                !s_hasReceivedPayout[groupId][member]
                    && (winner == address(0) || s_payoutPositions[groupId][me
mber] == selectedGroup.currentPayoutIndex)
            ) {
                winner = member;
                break;
            }
        }

        if (winner == address(0)) {
```

```
            for (uint256 i = 0; i < totalMembers; i++) {
                address member = selectedGroup.members[i];
                if (!s_hasReceivedPayout[groupId][member]) {
                    winner = member;
                    break;
                }
            }
        }

        require(winner != address(0), "Komiti: No eligible winner found");
        uint256 payoutAmount = (selectedGroup.perShareAmount * selectedGroup.
members.length);

        if (s_jointContributorsDetails[groupId][winner].primaryContributor !=
address(0)) {
            distributeJointPayout(groupId, winner, payoutAmount);
        } else {
            payable(winner).transfer(payoutAmount);
        }

        s_hasReceivedPayout[groupId][winner] = true;
        selectedGroup.currentPayoutIndex = (selectedGroup.currentPayoutIndex
+ 1) % totalMembers;
        emit Komiti__FundsDistributed(groupId, winner, payoutAmount);
    }
```

`distributeJointPayout()`

```
  function distributeJointPayout(uint256 groupId, address primaryContributor,
uint256 totalAmount) internal {
        // Group storage group = s_groups[groupId];
        JointContributors storage jointDetails = s_jointContributorsDetails[g
roupId][primaryContributor];

        uint256 totalContributions = s_contributions[groupId][primaryContribu
tor];
        uint256 primaryShare = totalAmount;

        for (uint256 i = 0; i < jointDetails.secondaryContributors.length; i+
+) {
            address secondary = jointDetails.secondaryContributors[i];
            uint256 secondaryContribution = s_contributionShares[groupId][sec
ondary];

            if (secondaryContribution > 0) {
                uint256 secondaryShare = (totalAmount * secondaryContribution
) / totalContributions;
                payable(secondary).transfer(secondaryShare);
```

```
                primaryShare -= secondaryShare;

                emit Komiti__JointPayoutDistributed(groupId, primaryContributor, secondary, secondaryShare);
            }
        }

        payable(primaryContributor).transfer(primaryShare);
    }
```

If a smart-contract address is granted a `USER_ROLE`, and joins the group as a member, there is a risk that the distribution of funds will lead to the Denial of Service if this smart-contract becomes the winner.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

## Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 1/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Info

## Recommendations

**Remediation:** It is recommended to use built-in `call()` function instead of `transfer()` to transfer native assets. This method does not impose a gas limit, it provides greater flexibility and compatibility with contracts having more complex business logic upon receiving the native tokens. When using the `call()` function, always ensure its execution is successful by checking the returned boolean value.

**Resolution:** Fixed in `1829f31`. All `transfer()` calls were refactored and replaced with low-level `call()` throughout the codebase.

## F-2025-14223 - Missing Empty-Length Array Validation in Loop-Based Functions - Info

**Description:**

Several functions in the `Komiti` contract iterate over user-provided arrays without validating that the array length is greater than zero.

Affected functionality:

functions `assignUserRoles()` and `revokeUserRoles()`

```
 function assignUserRoles(address[] memory users) external onlyRole(MANAGER_R
OLE) {
        for (uint256 i = 0; i < users.length; i++) {
            _grantRole(USER_ROLE, users[i]);
        }
    }

 function revokeUserRoles(address[] memory users) external onlyRole(MANAGER_R
OLE) {
        for (uint256 i = 0; i < users.length; i++) {
            _revokeRole(USER_ROLE, users[i]);
        }
    }
```

functions `assignOrganizerRoles()` and `revokeOrganizerRoles()`

```
 function assignOrganizerRoles(address[] memory users) external onlyRole(MANA
GER_ROLE) {
        for (uint256 i = 0; i < users.length; i++) {
            _grantRole(ORGANIZER_ROLE, users[i]);
        }
    }

 function revokeOrganizerRoles(address[] memory users) external onlyRole(MANA
GER_ROLE) {
        for (uint256 i = 0; i < users.length; i++) {
            _revokeRole(ORGANIZER_ROLE, users[i]);
        }
    }
```

function `setPayoutPositions()`

```
 ..
for (uint256 i = 0; i < users.length; i++) {
                s_payoutPositions[groupId][users[i]] = i;
```

```
                emit Komiti__PayoutPositionAssigned(groupId, users[i], i);
            }
..
```

Although an empty array will not cause a direct revert, it can cause **silent no-op executions**, leading to:

- **Unexpected behavior** (e.g., calling a role-assignment function but assigning roles to nobody).
- **Misconfiguration** when admin assumes a state change occurred, but the function executed zero iterations.
- **Broken invariants**, such as payout ordering relying on a non-empty array.
- **Potential misuse** where external integrations expect at least one element.

Failing to validate array length before looping is a known anti-pattern and should be corrected.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

## Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Info

## Recommendations

**Remediation:** Add an explicit validation check to the aforementioned functions to ensure that the looped arrays are not empty.

**Resolution:** Fixed in `1829f31`. The empty array length protection checks were implemented in the aforementioned functions.

## F-2025-14225 - Missing Zero Address Validation - Info

**Description:** The `Komiti` smart-contract lacks validation checks for zero addresses. Specifically:

- the constructor function lacks validation for the `admin` address allowing it to be `address(0)`:

```
constructor(address admin) {
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(ADMIN_ROLE, admin);
    }
```

- the `assignUserRoles()`, `assignOrganizerRoles()`, `setPayoutPositions()` lack validation for the members of the input `users` arrays they take as arguments:

```
function assignUserRoles(address[] memory users) external onlyRole(MANAGER_ROLE){}
function assignOrganizerRoles(address[] memory users) external onlyRole(MANAGER_ROLE){}
function setPayoutPositions(uint256 groupId, address[] calldata users) external checkIfGroupExit(groupId) onlyRole(ADMIN_ROLE) {}
```

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** Implement the following address validations:

- In the constructor of `Komiti` contract, add a check to ensure that `admin` address cannot be address(0).
- In the functions `assignUserRoles()`, `assignOrganizerRoles()`, `setPayoutPositions()` introduce a zero address(0) validation for the members of the input `users` parameters.

**Resolution:** Fixed in `1829f31`. Missing address zero validation checks were implemented in the aforementioned functions.

## F-2025-14227 - Incorrect Modifier Naming Reduces Code Clarity and Increases Risk of Misuse - Info

**Description:**

The modifier responsible for validating the existence of a Komiti group is incorrectly named as `checkIfGroupExit`, which semantically suggests "exit" rather than "exists".

While this issue does not alter runtime behavior, it introduces **semantic ambiguity**, decreases code readability, and increases the likelihood of **developer misinterpretation or misuse** during future maintenance, refactoring, or extension of the protocol.

```
modifier checkIfGroupExit(uint256 groupId) {
        Group storage group = s_groups[groupId];
        require(group.startTime != 0, "Komiti: group does not exist");
        _;
}
```

**Impact:**

- **Developer misunderstanding:** Incorrect expectations about what the modifier enforces may lead to incorrect usage in new functions.
- **Reduced auditability:** Future reviewers must spend additional time interpreting intent, increasing the likelihood of logic bugs slipping through.
- **Potential future naming conflicts:** If "exit" logic is ever added, the modifier name becomes misleading or obstructive.
- **Weaker guarantees for invariant enforcement:** Misnamed modifiers increase the risk of the wrong invariant being applied in the wrong context.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Info

## Recommendations

**Remediation:** It is recommended to rename the modifier to reflect its true purpose, such as:

```
modifier checkIfGroupExists(uint256 groupId) {
      Group storage group = s_groups[groupId];
      require(group.startTime != 0, "Komiti: group does not exist");
      _;
  }
```

**Resolution:** Fixed in `1829f31`. The affected modifier was replaced with `checkIfGroupExists` to reflect its true purpose.

## F-2025-14234 - Absence of Custom Errors Leading to Increased Gas Costs - Info

**Description:**

Starting with Solidity version 0.8.4, a new feature called "custom errors" was introduced. This allows developers to define specific error conditions in a clearer, more meaningful way without the need for string-based error messages. Before this version, developers commonly used `require` statements with strings to signal validation failures or specific conditions. However, including unique strings for revert reasons adds to gas costs, making transactions more expensive.

**Other affected functions:**

- `checkIfGroupExit()`
- `createGroup()`
- `joinGroup()`
- `joinGroupWithJointContributor()`
- `acceptInviteForJointContributor()`
- `returnFunds()`
- `updateKomiti()`
- `startKomiti()`
- `setPayoutPositions()`
- `contribute()`
- `contributeAsJointMember()`
- `distributeFunds()`

In contrast, custom errors are defined by their name and the types of their parameters, without the need for string data storage. As a result, using custom errors instead of string-based `require` messages reduces gas consumption, making contracts more efficient.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Info

## Recommendations

**Remediation:** To enhance the gas efficiency of the code, it is recommended to replace traditional `require` statements with custom errors. For instance:

from:

```
require(!group.isPayoutStarted, "Komiti: Cannot join after payout starts");
```

to:

```
error PayoutStarted();
if (group.isPayoutStarted) {
    revert PayoutStarted();
}
```

Custom errors are more cost-effective because they reduce the amount of data that needs to be stored and processed on the blockchain when handling exceptions.

**Resolution:** In commit `1829f31`, the contract has replaced gas-expensive `require` statements with string messages by defining and using Solidity custom errors.

## F-2025-14258 - Violation of Solidity Naming Conventions in Events Reduces Code Quality and Readability - Info

**Description:**

The contract contains multiple instances where naming conventions violate the Solidity Style Guide.

These violations do not break functionality but **reduce readability** and **code quality**.
Following the Solidity Style Guide is crucial for readability and correctness expectations across the ecosystem.
**Affected Functionality:**

- `event Komiti__GroupStarted()`
- `event Komiti__GroupDeleted()`
- `event Komiti__UserJoined()`
- `event Komiti__GroupUpdated()`
- `event Komiti__FundsDistributed()`
- `event Komiti__ContributionMade()`
- `event Komiti__PayoutPositionAssigned()`
- `event Komiti__GroupCreated()`
- `event Komiti__JointContributorAdded()`
- `event Komiti__JointContributorInviteAccepted()`
- `event Komiti__JointPayoutDistributed()`
- `event Komiti__JointContributionMade()`
- `event Komiti__JointContributorInviteReturned()`

Events should be named using CapWords style; however, the existing event names include double underscores, which violates the standard. According to the Solidity Style Guide, the underscores can be used for non-external functions and variables, constants or to avoid collision if there are variables with the same naming.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Dependent

**Severity:** Info

## Recommendations

**Remediation:** It is recommended to rename all events to pure PascalCase and remove double underscores as follows:

```
Komiti__GroupCreated -> GroupCreated
Komiti__UserJoined -> UserJoined
Komiti__FundsDistributed -> FundsDistributed
```

**Resolution:** Fixed in **1829f31**. All the aforementioned events have been properly renamed following the Solidity Style Guide.

## F-2025-14293 - Integer Division Precision Loss Causes Underfunded Joint Contributions - Info

**Description:**

The joint contribution flow allows two participants (a primary and a secondary contributor) to collectively satisfy the `perShareAmount` requirement by each paying **half** of the required share.

Both `joinGroupWithJointContributor()` and `acceptInviteForJointContributor()` enforce the following validation:

```
require(msg.value >= group.perShareAmount / 2, "Komiti: Incorrect amount sent
");
```

However, Solidity performs **integer division with truncation**, which introduces a precision loss when `perShareAmount` is an odd number.

For example:

```
perShareAmount = 83; perShareAmount / 2 = 41 (integer truncation)
If both contributors will pay 41, then the perShareAmount will be 82, not 83
as expected.
Hence 1 unit is silently lost.
```

The protocol incorrectly assumes that the joint contribution always sums to `perShareAmount`, but due to integer truncation, this invariant does not hold for small odd values.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 4/5

**Exploitability:** Semi-Dependent

**Complexity:** Medium

**Severity:** Info

### Recommendations

**Remediation:** There are several safe and robust ways to address this issue:

- consider tracking the contribution of the primary user and then utilize it in the require statement in the `acceptInviteForJointContributor()` function as follows:

```
require(
    msg.value + s_contributions[groupId][primaryContributor] >= group.perShar
eAmount,
    "Komiti: Joint contribution insufficient"
);
```

- disallow integer truncation entirely by enforcing:

```
require(msg.value * 2 == group.perShareAmount, "Komiti: Invalid joint contrib
ution split");
```

**Resolution:** Fixed in `1829f31`. The `joinGroupWithJointContributor()` and `acceptInviteForJointContributor()` functions which were affected by the precision loss have been removed from the codebase.

## F-2025-14299 - Lack of Mutual Exclusion Allows Role Overlap Risks - Info

**Description:**

The contract defines distinct roles: `USER_ROLE`, `MANAGER_ROLE`, `ORGANIZER_ROLE`, and `ADMIN_ROLE`. These roles have different capabilities: Users contribute, Organizers create groups, Managers assign roles.

The contract does not enforce mutual exclusion between roles. An address can simultaneously be a User, Manager, Organizer, and Admin. While sometimes necessary, unrestricted role overlap concentrates power and risks "super-user" scenarios where a single compromised account can bypass separation-of-duties controls. For example, a Manager can assign themselves `USER_ROLE` to participate, or an Organizer can assign themselves `USER_ROLE` to fill slots in their own group, effectively becoming a centralized operator of a "community" group.

```
function assignUserRoles(address[] memory users) external onlyRole(MANAGER_RO
LE) {
    _grantRole(USER_ROLE, users[i]);
}
```

Similarly for `assignOrganizerRoles`. There is no logic preventing `grantRole(ORGANIZER_ROLE, userAddress)` where `hasRole(USER_ROLE, userAddress)` is `true`.

This can lead to a single entity can consolidate all roles, negating the governance benefits of role-based access control.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Accepted

### Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 1/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** Define clear separation of duties. If an Organizer should not be a User in their own group, enforce `require(!hasRole(ORGANIZER_ROLE, account))` when granting `USER_ROLE`, or verify at the group level.

**Resolution:** `Accepted`. The client is aware of the finding and accepted the risk.

## F-2025-14300 - Operational Lockout Risk Due to Missing Initial Manager Assignment - Info

**Description:**

The `Komiti` contract relies on role-based access control to manage permissions. Specifically, the `MANAGER_ROLE` is required to call `assignUserRoles`, which in turn grants the `USER_ROLE`. Only addresses with `USER_ROLE` can join groups or contribute funds (except for the initial group creator). The `ADMIN_ROLE` and `DEFAULT_ADMIN_ROLE` are assigned to the deployer in the `constructor`.

The contract `constructor` initializes the `ADMIN_ROLE` but fails to assign the `MANAGER_ROLE` to any address. Without a `MANAGER_ROLE` assigned, the `assignUserRoles` function is uncallable. Since normal users cannot interact with the protocol without `USER_ROLE` (which must be granted by a Manager), the contract is deployed in a semi-functional state where user onboarding is blocked until the Admin manually intervenes to grant the Manager role.

```
constructor(address admin) {
    _grantRole(DEFAULT_ADMIN_ROLE, admin);
    _grantRole(ADMIN_ROLE, admin);
}
```

The `MANAGER_ROLE` is defined but never granted. The function `assignUserRoles` is protected by `onlyRole(MANAGER_ROLE)`. While the admin (holding `DEFAULT_ADMIN_ROLE`) can grant the `MANAGER_ROLE` post-deployment, the lack of initial assignment creates a risk of operational lockout if the deployment process assumes a "ready-to-use" state. If the Admin is a DAO or multisig with a time-lock, this oversight delays the protocol's launch or user onboarding. Additionally, if the Admin key is lost or the governance process fails before assigning a Manager, the ability to onboard new users via the intended `assignUserRoles` flow is permanently lost. Hence it is possible that the protocol cannot onboard users immediately after deployment.

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 2/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

## Recommendations

**Remediation:** Grant the `MANAGER_ROLE` to the admin (or a dedicated manager address) in the constructor to ensure the contract is fully operational upon deployment.

**Resolution:** Fixed in `1829f31`. The `MANAGER_ROLE` is now granted to the admin address in the constructor function.

## F-2025-14301 - Floating Pragma - Info

**Description:** In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.20`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The project currently uses floating pragma declarations ( `^0.8.20`) in its Solidity contracts. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

**Assets:**

- contracts/RYTStablecoin.sol [ryt-stablecoin-develop.zip]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** It is recommended to **lock the pragma version** to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates.

**Resolution:** Fixed in SHA3-256 `04264b6`. The floating pragma was locked to 0.8.20.

## F-2025-14302 - Redundant Imports and Unused Error Definitions Increase Gas Usage - Info

**Description:**

The `RYTStablecoin` contract includes **unused imports** and **declared but never used custom errors**, which unnecessarily increase code complexity and reduce overall clarity.
While these issues do not directly introduce exploitable vulnerabilities, they negatively impact maintainability, auditability, and developer trust, especially for a security-sensitive contract such as a stablecoin.

The following components are affected:

**1. Redundant / Unused Imports**

In `RYTStablecoin` contract the following imports are declared but never used anywhere in the contract logic:

```
import "@openzeppelin/contracts/utils/Address.sol";
```

The `Address` library is not referenced directly or indirectly in the contract. Its inclusion provides no functional benefit and should be removed.

**2. Unused Custom Error Definitions**

The contract defines several custom errors that are never reverted against:

```
error InvalidToken(address token);
```

This error is never triggered by any revert statement or conditional branch. Its presence suggests either: Incomplete implementation, or Legacy code that was partially refactored but not cleaned up.

Unused errors create ambiguity about intended behavior and complicate reasoning about failure modes.

**Assets:**

- contracts/RYTStablecoin.sol [ryt-stablecoin-develop.zip]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Info

## Recommendations

**Remediation:** It is recommended to:

- **remove unused imports**
  Delete unused imports from the contract:

```
// Remove this line
import "@openzeppelin/contracts/utils/Address.sol";
```

- **remove or implement unused errors**
  either:
  - remove unused error definitions entirely, or
  - implement them properly where intended (e.g., token validation logic if InvalidToken was meant to be used).

```
// Remove if not used
error InvalidToken(address token);
```

- **add a CI/Linting Rule**
  Consider enforcing:
  - Solidity linter rules
  - Static analysis (e.g., Slither, Solhint)

to prevent unused imports and errors from being merged in future revisions.

**Resolution:** Fixed in SHA3-256 `04264b6`. Redundant imports and unused error definitions have been removed from the codebase.

## F-2025-14305 - Unchecked Return Values From Functions May Cause Silent State Inconsistencies - Info

**Description:**

Throughout the contracts, multiple external calls and internal helper invocations return values that are never validated or checked. This pattern can cause the contract to proceed with an incorrect or partially completed state, without reverting or signaling that an operation has failed.

For instance, the return values from internal functions such as `_grantRole()` and `_revokeRole()` allow the contract to assume success even when the downstream call reverted internally but returned a benign result (such as false). Over time, this may cause divergence between expected and actual states across modules.

Internal functions `_grantRole()` and `_revokeRole()` can be found in the code snippets below:

```
function _grantRole(bytes32 role, address account) internal virtual returns (
bool) {
        if (!hasRole(role, account)) {
            _roles[role].hasRole[account] = true;
            emit RoleGranted(role, account, _msgSender());
            return true;
        } else {
            return false;
        }
    }
```

```
function _revokeRole(bytes32 role, address account) internal virtual returns
(bool) {
        if (hasRole(role, account)) {
            _roles[role].hasRole[account] = false;
            emit RoleRevoked(role, account, _msgSender());
            return true;
        } else {
            return false;
        }
    }
```

The return values of the functions above are not validated in the public functions below:

`RYTStablecoin::constructor()`

```
constructor(
        string memory name_,
```

```
        string memory symbol_,
        uint8 decimals_,
        address admin,
        uint256 cap
    )
        ERC20(name_, symbol_)
        ERC20Permit(name_)
    {
        if (admin == address(0)) revert ZeroAddress();

        _decimalsCustom = decimals_;
        maxSupply = cap;

        // Grant initial roles to the provided admin.
        _grantRole(ADMIN_ROLE, admin);
        _grantRole(MINTER_ROLE, admin);
        _grantRole(PAUSER_ROLE, admin);
        _grantRole(BLOCKLISTER_ROLE, admin);
        _grantRole(RESCUER_ROLE, admin);
    }
```

`Komiti::constructor()`

```
 constructor(address admin) {
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(ADMIN_ROLE, admin);
    }
```

Functions `assignUserRoles()/revokeUserRoles()` and functions `assignOrganizerRoles()/revokeOrganizerRoles()` of the `Komiti` contract.

```
 function assignUserRoles(address[] memory users) external onlyRole(MANAGER_R
OLE) {
        for (uint256 i = 0; i < users.length; i++) {
            _grantRole(USER_ROLE, users[i]);
        }
    }

 function revokeUserRoles(address[] memory users) external onlyRole(MANAGER_R
OLE) {
        for (uint256 i = 0; i < users.length; i++) {
            _revokeRole(USER_ROLE, users[i]);
        }
    }

 function assignOrganizerRoles(address[] memory users) external onlyRole(MANA
GER_ROLE) {
        for (uint256 i = 0; i < users.length; i++) {
```

```
            _grantRole(ORGANIZER_ROLE, users[i]);
        }
    }

 function revokeOrganizerRoles(address[] memory users) external onlyRole(MANA
GER_ROLE) {
        for (uint256 i = 0; i < users.length; i++) {
            _revokeRole(ORGANIZER_ROLE, users[i]);
        }
```

**Assets:**

- contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract]
- contracts/RYTStablecoin.sol [ryt-stablecoin-develop.zip]

**Status:** Accepted

## Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Info

## Recommendations

**Remediation:** It is recommended to validate the return values of the aforementioned functions and revert if a function returns false as follows:

```
 error RoleNotGranted();
error RoleNotRevoked();
...
require(_grantRole(USER_ROLE, users[i]), RoleNotGranted());
...

or
...
 require(_revokeRole(USER_ROLE, users[i]), RoleNotRevoked());
...
```

**Resolution:** `Accepted`. The finding is partially fixed for the affected contracts, hence the client accepted the risk.

# Disclaimers

## Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

## Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

# Appendix 1. Definitions

## Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

| Severity | Description |
|---|---|
| Critical | Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation. |
| High | High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation. |
| Medium | Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category. |
| Low | Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution. |

## Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

# Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

| Scope Details | |
|---|---|
| 1st Repository | https://github.com/ryt-io/Komitte-Contract |
| Initial Commit | 52685f90e291cc6e9be3a5a4bba052fe2f940164 |
| Final Commit | 1829f31e73312ff587f102cc2246d3a646982195 |
| 2nd Repostiory | ryt-stablecoin-develop.zip |
| Initial fille SHA3-256 | 851798d95a8fa81a37c950c9c2da0a4085a96f52e73d38eb2f044b56868adb21 |
| Final file SHA3-256 | 04264b6db0b5fdf81a83d00c9cd685e2d715a9af6ebdc33d30c95755e006ac56 |
| Whitepaper | N/A |
| Requirements | README.md |
| Technical Requirements | README.md |

| Asset | Type |
|---|---|
| contracts/Komiti.sol [https://github.com/ryt-io/Komitte-Contract] | Smart Contract |
| contracts/RYTStablecoin.sol [ryt-stablecoin-develop.zip] | Smart Contract |

# Appendix 3. Additional Valuables

## Verification of System Invariants

During the audit of `Komiti+RYTStablecoin`, Hacken followed its methodology by performing fuzz-testing on the project's main functions. Foundry, a tool used for fuzz-testing, was employed to check how the protocol behaves under various inputs. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use fuzz-testing to ensure that several system invariants hold true in all situations.

Fuzz-testing allows the input of many random data points into the system, helping to identify issues that regular testing might miss. A specific Echidna fuzzing suite was prepared for this task, and throughout the assessment, 6 invariants were tested over 120000 runs. This thorough testing ensured that the system works correctly even with unexpected or unusual inputs.

| Invariant | Test Result | Run Count |
|---|---|---|
| `Komiti::assignUserRoles()` should always assign the `USER_ROLE` to unique users. | Passed | 20000 |
| `Komiti::revokeUserRoles()` should always revoke the `USER_ROLE` from unique users. | Passed | 20000 |
| `Komiti::assignOrganizerRoles()` should always assign the `USER_ROLE` to unique users. | Passed | 20000 |
| `Komiti::revokeOrganizerRoles()` should always revoke the `USER_ROLE` from unique users. | Passed | 20000 |
| `Komiti::createGroup()` should always increase the `s_groupCounter` value by 1. | Passed | 20000 |
| `Komiti::joinGroup()` should always increase the length of the group members list for unique contributors. | Passed | 20000 |

## Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

## Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.